



RAJALAKSHMI ENGINEERING COLLEGE

MOBILE APPLICATION DEVELOPMENT

Laboratory Record Note Book

NAME AKSHAYAA M

BRANCH B TECH INFORMATION TECHNOLOGY

UNIVERSITY REGISTER No. 2116231001010

COLLEGE ROLL No. 231001010

SEMESTER VI

ACADEMIC YEAR 2025 -26



**RAJALAKSHMI
ENGINEERING
COLLEGE**

BONAFIDE CERTIFICATE

NAME AKSHAYAA M

ACADEMIC YEAR 2025 -26 SEMESTER VI BRANCH..... B TECH IT

UNIVERSITY REGISTER No.

2116231001010

Certified that this is the bonafide record of work done by the above student in the

MOBILE APPLICATION
DEVELOPMENT

Laboratory during the year 20 25 - 20 26

Signature of Faculty - in - Charge

Submitted for the Practical Examination held on

Internal Examiner

External Examiner

TABLE OF CONTENTS

S.NO.	DATE	EXPERIMENT TITLE	MARK	SIGN.
1.	05-01-2026	Kotlin Basics		
2.	19-01-2026	Functions		
3	29-01-2026	Classes and Objects		
4	05-02-2026	Build First Android App		
5	16-02-2026	Layouts		
6	26-02-2026	App Navigation		
7	05-03-2026	Activity and Fragment Life Cycle		
8	12-03-2026	App Architecture (UI LAYER)		
9	16-03-2026	App Architecture (Persistence)		
10	30-03-2026	Advanced Recycler View Use cases		
11	06-04-2026	Connect to the Internet		
12	16-04-2026	Repository Pattern and WorkManager		
13.	27-04-2026	App UI Design		

INTRODUCTION

What is Kotlin?

Kotlin is a modern, trending programming language that was released in 2016 by JetBrains. It has become very popular since it is compatible with [Java](#) (one of the most popular programming languages out there), which means that Java code (and libraries) can be used in Kotlin programs.

Kotlin is used for:

- Mobile applications (specially Android apps)
- Web development
- Server side applications
- Data science

Why Use Kotlin?

- Kotlin is fully compatible with Java
- Kotlin works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc.)
- Kotlin is concise and safe
- Kotlin is easy to learn, especially if you already know Java
- Kotlin is free to use
- Big community/support

Kotlin IDE

The easiest way to get started with Kotlin, is to use an IDE.

An IDE (Integrated Development Environment) is used to edit and compile code.

In this chapter, we will use IntelliJ (developed by the same people that created Kotlin) which is free to download from <https://www.jetbrains.com/idea/download/>.

Android's Kotlin-first approach

At Google I/O 2019, we announced that Android development will be increasingly Kotlin-first, and we've stood by that commitment. Kotlin is an expressive and concise programming language that reduces common code errors and easily integrates into existing apps. If you're looking to build an Android app, we recommend starting with Kotlin to take advantage of its best-in-class features.

In an effort to support Android development using Kotlin, we co-founded the [Kotlin Foundation](#) and have ongoing investments in improving compiler performance and build speed. To learn more about Android's commitment to being Kotlin-first, see [Android's commitment to Kotlin](#).

Why is Android development Kotlin-first?

We reviewed feedback that came directly from developers at conferences, our Customer Advisory Board (CAB), Google Developer Experts (GDE), and through our developer research. Many developers already enjoy using Kotlin, and the request for more Kotlin support was clear. Here's what developers appreciate about writing in Kotlin:

- **Expressive and concise:** You can do more with less. Express your ideas and reduce the amount of boilerplate code. 67% of professional developers who use Kotlin say Kotlin has increased their productivity.
- **Safer code:** Kotlin has many language features to help you avoid common programming mistakes such as null pointer exceptions. Android apps that contain Kotlin code are 20% less likely to crash.
- **Interoperable:** Call Java-based code from Kotlin, or call Kotlin from Java-based code. Kotlin is 100% interoperable with the Java programming language, so you can have as little or as much of Kotlin in your project as you want.
- **Structured Concurrency:** Kotlin coroutines make asynchronous code as easy to work with as blocking code. Coroutines dramatically simplify background task management for everything from network calls to accessing local data.

What does Kotlin-first mean?

When building new Android development tools and content, such as Jetpack libraries, samples, documentation, and training content, we will design them with Kotlin users in mind while continuing to provide support for using our APIs from the Java programming language.

Components

Components of an Android Application

Introduction to Activities in Android

Services in Android with Example

Content Providers in Android with Example

Broadcast Receiver in Android With Example

TextView

TextView in Kotlin

Dynamic TextView in Kotlin

AutoCompleteTextView in Kotlin

Dynamic AutoCompleteTextView in Kotlin

CheckedTextView in Kotlin

Dynamic CheckedTextView in Kotlin

ScrollView

ScrollView in Android

HorizontalScrollView in Kotlin

Dynamic ScrollView in Kotlin

DynamicHorizontal ScrollView in Kotlin

Other Views

Dynamic ImageView in Kotlin
Android ListView in Kotlin
CardView in Android With Example
VideoView in Kotlin
Zigzag View in Android
ViewAnimator in Android with Example

Button

Button in Kotlin
RadioButton in Kotlin
ImageButton in Kotlin
ToggleButton in Kotlin
RadioGroup in Kotlin
CheckBox in Kotlin
Dynamic ImageButton in Kotlin
Dynamic RadioButton in Kotlin

EditText

Android EditText in Kotlin
Dynamic EditText in Kotlin

Layouts

Layouts in Android UI Design
Android UI Layouts
Android TableLayout in Kotlin
Android FrameLayout in Kotlin
Android RelativeLayout in Kotlin
Android ConstraintLayout in Kotlin
Android LinearLayout in Kotlin
Android Motion Layout in Kotlin

Bar

SeekBar in Kotlin
Dynamic SeekBar in Kotlin
Discrete SeekBar in Kotlin
RatingBar in Kotlin
Dynamic RatingBar in Kotlin
ProgressBar in Kotlin
Dynamic ProgressBar in Kotlin

Switcher

Switch in Kotlin
Dynamic Switch in Kotlin
TextSwitcher in Kotlin
Dynamic TextSwitcher in Kotlin
ImageSwitcher in Kotlin
Dynamic ImageSwitcher in Kotlin
Time and Date Picker
TimePicker in Kotlin

How to add a Custom-styled Toast in Android using Kotlin

Misc

Spinner

Dynamic Spinner in Kotlin

Advantage of Kotlin for Android Development

Choosing Kotlin for Android development offers several compelling benefits:

Concise Code: Kotlin lets you write less code to do the same things, making it easier to read and maintain.

Null Safety: Kotlin helps prevent common errors by making it harder to accidentally use null values, which can crash your app.

Interoperability with Java: You can use Kotlin and Java together in the same project.

Kotlin works well with existing Java code, so you don't have to rewrite everything.

Modern Features: Kotlin has many modern programming features that make development faster and more enjoyable, like extension functions and data classes.

Better Performance: Kotlin often results in better-performing apps due to its efficient syntax and features.

Official Support: Kotlin is officially supported by Google for Android development, ensuring good documentation and ongoing improvements.

Active Community: A growing community of developers means lots of resources, libraries, and tools to help you learn and solve problems.

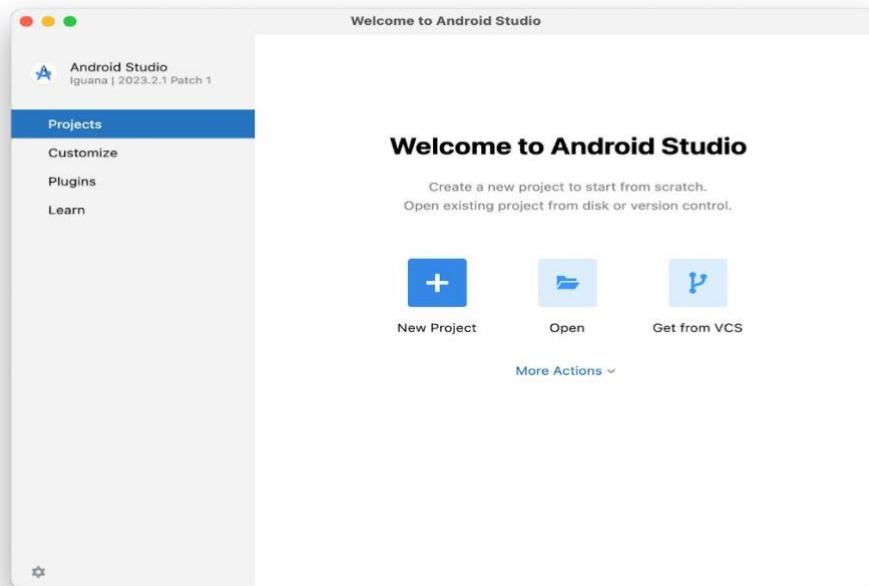
codelab, you create an Android app with the **Empty Activity** project template provided by Android Studio.

To create a project in Android Studio:

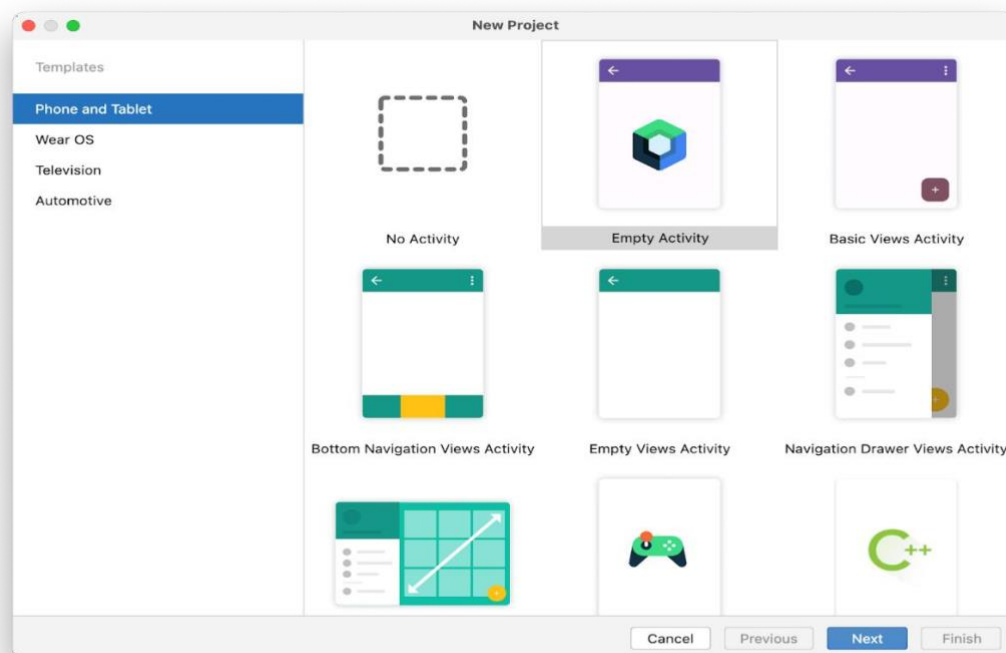
1. Double click the Android Studio icon to launch Android Studio.



2. In the **Welcome to Android Studio** dialog, click **New Project**.



The **New Project** window opens with a list of templates provided by Android Studio.



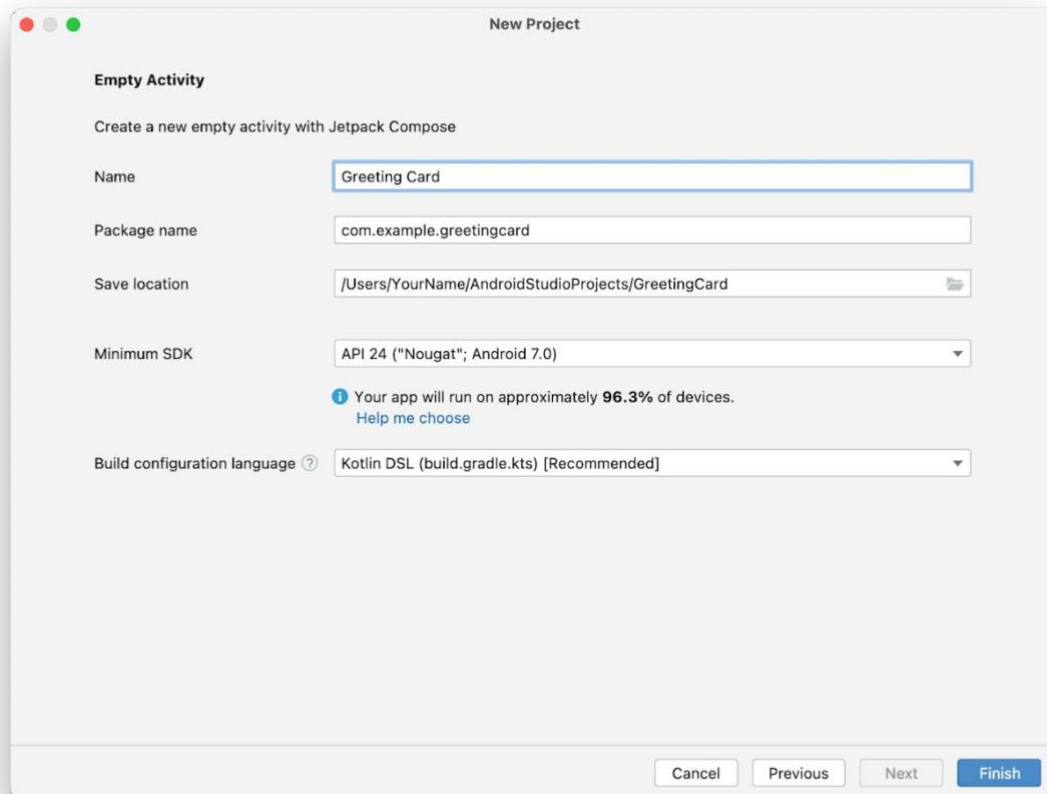
In Android Studio, a project template is an Android project that provides the blueprint for a certain type of app. Templates create the structure of the project and the files needed for Android Studio to build your project. The template that you choose provides starter code to get you going faster.

3. Make sure the **Phone and Tablet** tab is selected.
4. Click the **Empty Activity** template to select it as the template for your project.
The **Empty Activity** template is the template to create a simple project that you can use to build a Compose app. It has a single screen and displays the text "Hello Android!".
5. Click **Next**. The **New Project** dialog opens. This has some fields to configure your project.
6. Configure your project as follows:

The **Name** field is used to enter the name of your project, for this codelab type "Greeting Card".

Leave the **Package name** field as is. This is how your files will be organized in the file structure. In this case, the package name will be com.example.greetingcard. Leave the **Save location** field as is. It contains the location where all the files related to your project are saved. Take a note of where that is on your computer so that you can find your files.

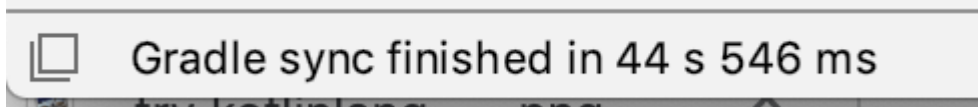
Select **API 24: Android 7.0 (Nougat)** from the menu in the **Minimum SDK** field. [Minimum SDK](#) indicates the minimum version of Android that your app can run on.



7. Click **Finish**. This may take a while - this is a great time to get a cup of tea! While Android Studio is setting up, a progress bar and message indicates whether Android Studio is still setting up your project. It may look like this:



A message that looks similar to this informs you when the project set up is created.



8. You may see a **What's New** pane which contains updates on new features in Android Studio. Close it for now.



What's New in Iguana

This panel describes some of the new features and behavior changes included in this update.

To open this panel again later, select **Help > What's New in Android Studio** from the main menu.

[Read in a browser](#)

Version control system integration in App Quality Insights

Use **App Quality Insights** to navigate from a Crashlytics stack trace to the relevant code—at the point in time when the crash happened. When you view crash reports, you can choose to navigate to the line of code in your current git checkout or view a diff between the current checkout and the version of your codebase that generated the crash.

[Learn more](#)

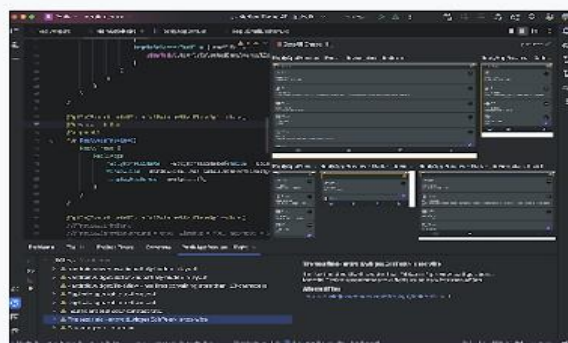
View Crashlytics crash variants in App Quality Insights

To help you analyze the root causes of a crash, use **App Quality Insights** to view crash reports by issue *variants*, or groups of events that share similar stack traces.

[Learn more](#)

UI Check Mode in Compose Preview

To help developers build more adaptive and accessible UI in Jetpack Compose, Android Studio Iguana Canary 5 introduced a new UI Check mode in Compose Preview. This feature works similar to [Visual Linting](#) and Accessibility checks integrations for View. To activate Compose UI check mode for Android Studio to automatically audit your Compose UI and check for adaptive and accessibility issues across different screen sizes, such as text stretched on large screens and low color contrast. The mode highlights issues found in different Preview configurations and lists them in the problems panel.



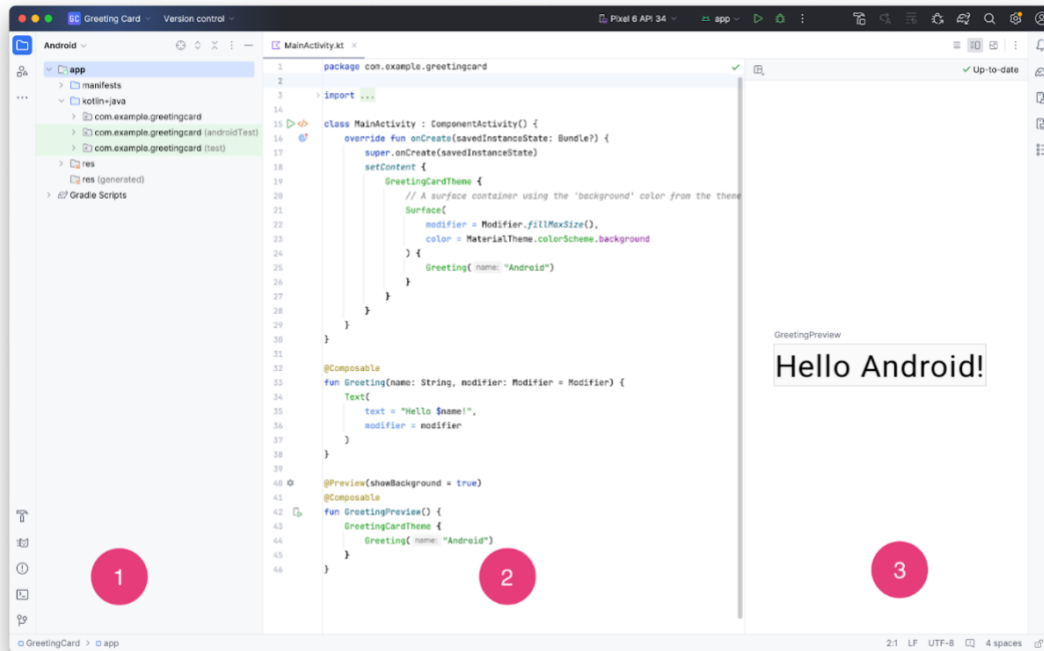
9. Click **Split** on the top right of Android Studio, this allows you to view both code and design. You can also click **Code** to view code only or click **Design** to view

design only.



Split

After pressing **Split** you should see three areas:



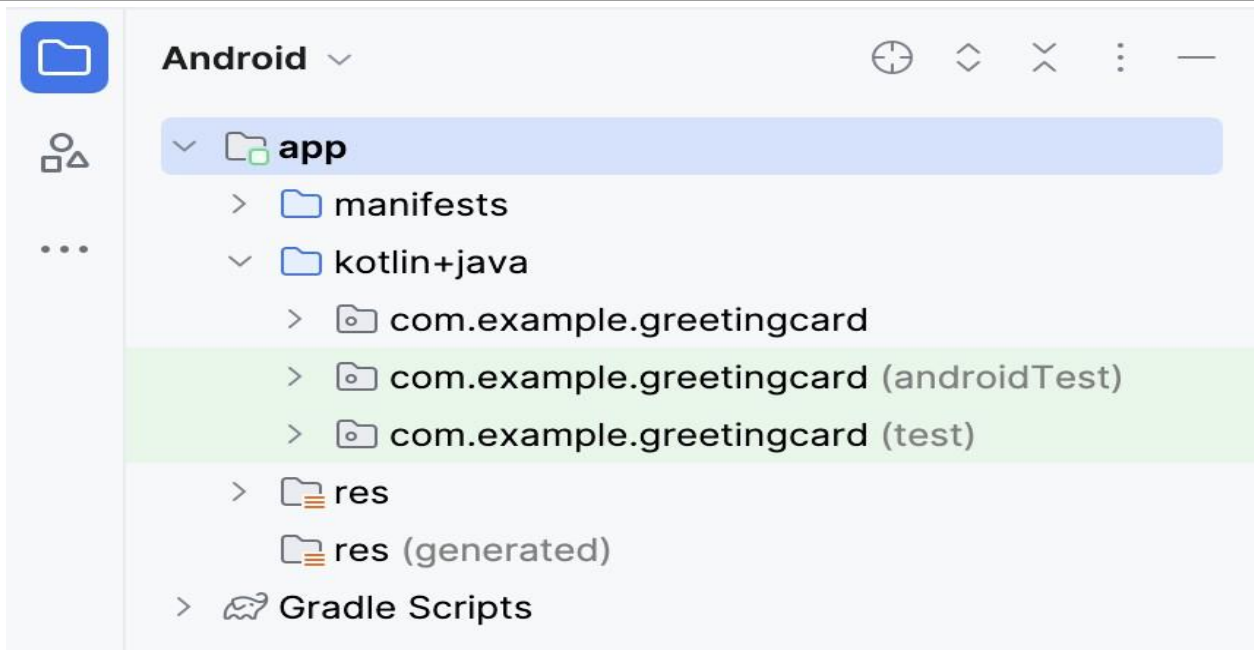
- The **Project** view (1) shows the files and folders of your project
- The **Code** view (2) is where you edit code
- The **Design** view (3) is where you preview what your app looks like

In the **Design** view, you may see a blank pane with this text:

10. Click **Build & Refresh**. It may take a while to build but when it is done the preview shows a text box that says "**Hello Android!**". Empty Compose activity contains all the code necessary to create this app.

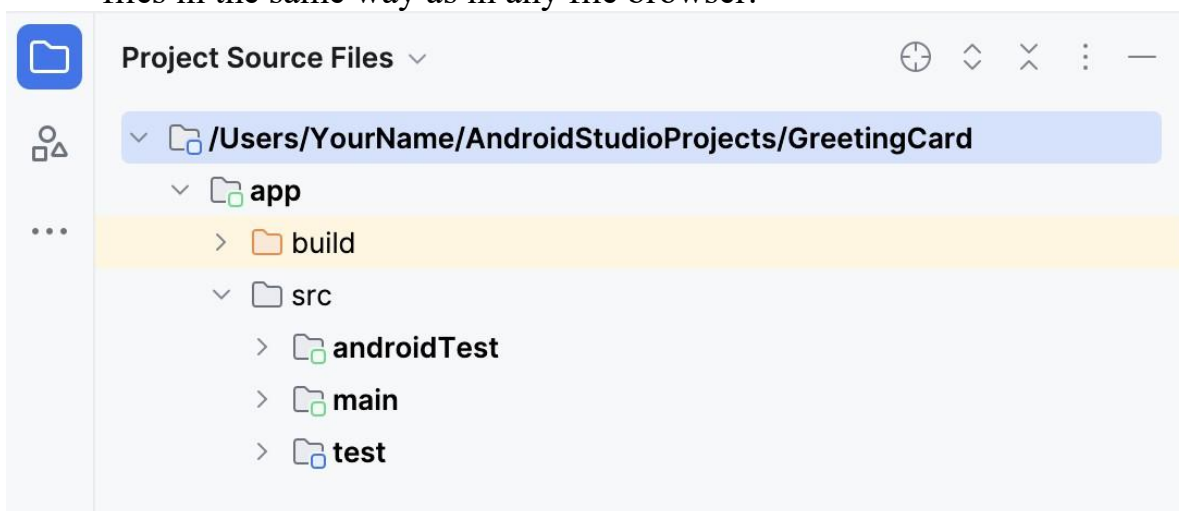
explore Android Studio by becoming familiar with the file structure.

1. In Android Studio, take a look at the **Project** tab. The **Project** tab shows the files and folders of your project. When you were setting up your project the package name was **com.example.greetingcard**. You can see that package right here in the **Project** tab. A package is basically a folder where code is located. Android Studio organizes the project in a directory structure made up of a set of packages.
2. If necessary, select **Android** from the drop-down menu in the **Project** tab.



This is the standard view and organization of files that you use. It's useful when you write code for your project because you can easily access the files you will be working on in your app. However, if you look at the files in a file browser, such as Finder or Windows Explorer, the file hierarchy is organized very differently.

3. Select **Project Source Files** from the drop-down menu. You can now browse the files in the same way as in any file browser.



4. Select **Android** again to switch back to the previous view. You use the **Android** view for this course. If your file structure ever looks strange, check to make sure you're still in **Android** view.

GreetingPreview

Hello Android!

Code view of the MainActivity.kt file. Notice there are some automatically generated functions in this code, specifically the onCreate() and the setContentView() functions.

Note: Remember that a function is a segment of a program that performs a specific task.

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
```

```

super.onCreate(savedInstanceState)
setContent {
    GreetingCardTheme {
        // A surface container using the 'background' color from the theme
        Surface(
            modifier = Modifier.fillMaxSize(),
            color = MaterialTheme.colorScheme.background
        ) {
            Greeting("Android")
        }
    }
}
}

```

The `onCreate()` function is the entry point to this Android app and calls other functions to build the user interface. In Kotlin programs, the `main()` function is the entry point/starting point of execution. In Android apps, the `onCreate()` function fills that role.

The [setContent\(\)](#) function within the `onCreate()` function is used to define your layout through composable functions. All functions marked with the `@Composable` annotation can be called from the `setContent()` function or from other Composable functions. The annotation tells the Kotlin compiler that this function is used by Jetpack Compose to generate the UI.

Note: The compiler takes the Kotlin code you wrote, looks at it line by line, and translates it into something that the computer can understand. This process is called compiling your code.

Next, look at the `Greeting()` function. The `Greeting()` function is a Composable function, notice the `@Composable` annotation above it. This Composable function takes some input and generates what's shown on the screen.

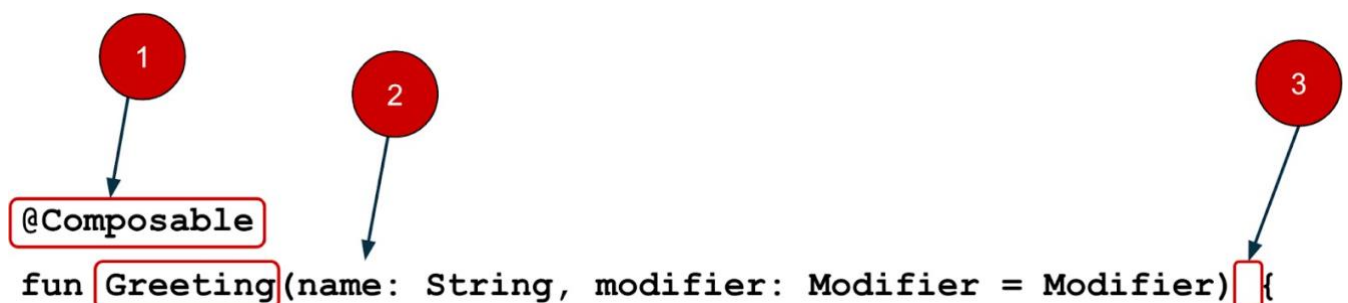
`@Composable`

```

fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}

```

You've learned about functions before (if you need a refresher, refer to the [Create and use functions in Kotlin codelab](#)), but there are a few differences with composable functions.



1. You add the `@Composable` annotation before the function.

-
- 2. `@Composable` function names are capitalized.
-
- 3. `@Composable` functions can't return anything.

`@Composable`

```
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}
```

Right now the `Greeting()` function takes in a name and displays Hello to that person.

1. Update the `Greeting()` function to introduce yourself instead of saying "Hello":

`@Composable`

```
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hi, my name is $name!",
        modifier = modifier
    )
}
```

2. Android should automatically update the preview.

GreetingPreview



Hi, my name is Android!

Great! You changed the text, but it introduces you as Android, which is probably not your name. Next, you will personalize it to introduce you with your name!

The `GreetingPreview()` function is a cool feature that lets you see what your composable looks like without having to build your entire app. To enable a preview of a composable, annotate with `@Composable` and `@Preview`. The `@Preview` annotation tells Android Studio that this composable should be shown in the design view of this file.

As you can see, the `@Preview` annotation takes in a parameter called `showBackground`. If `showBackground` is set to **true**, it will add a background to your composable preview. Since Android Studio by default uses a light theme for the editor, it can be hard to see the difference between `showBackground = true` and `showBackground = false`. However, this is an example of what the difference looks like. Notice the white background on the image set to `true`.

GreetingPreview



Hello Android!

Kotlin Basics

Aim

To study and implement basic Kotlin programming concepts such as variables, data types, operators, control statements, and null safety.

Algorithm

1. Start the Android Studio IDE.
2. Create a new Kotlin project.
3. Declare variables using val and var.
4. Use basic data types such as Int, Double, String, and Boolean.
5. Implement conditional statements (if, when).
6. Implement loops (for, while).
7. Display output using println() or UI components.
8. Run the program and verify output.

Coding

```
package com.example.basic1
```

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.compose.ui.tooling.preview.Preview
import com.example.basic1.ui.theme.Basic1Theme
```

```
// Function examples defined outside composable
```

```
fun add(a: Int, b: Int): Int = a + b
```

```
fun greet(name: String, greeting: String = "Hello"): String = "$greeting, $name!"
```

```
fun sumAll(vararg numbers: Int): Int = numbers.sum()
```

```
fun operate(a: Int, b: Int, operation: (Int, Int) -> Int): Int = operation(a, b)
```

```
tailrec fun factorial(n: Int, acc: Int = 1): Int = if (n <= 1) acc else factorial(n - 1, acc * n)
```

```
// Class examples defined outside composable
```

```
class Person(val name: String, var age: Int) {
```

```

    fun introduce(): String = "Hi, I'm $name and I'm $age years old"
}

data class User(val id: Int, val username: String, val email: String)

enum class Priority(val value: Int) {
    LOW(1), MEDIUM(2), HIGH(3), URGENT(4);
    fun getDisplayName(): String = name.lowercase().replaceFirstChar { it.uppercase() }
}

object AppConfig {
    const val APP_NAME = "Kotlin Demo"
    const val VERSION = "1.0.0"
    fun getConfig(): String = "$APP_NAME v$VERSION"
}

class MathUtils {
    companion object {
        const val PI = 3.14159
        fun circleArea(radius: Double): Double = PI * radius * radius
    }
}

abstract class Animal(val name: String) {
    abstract fun makeSound(): String
    fun sleep(): String = "$name is sleeping"
}

class Dog(name: String) : Animal(name) {
    override fun makeSound(): String = "$name says: Woof!"
}

interface Drawable {
    fun draw(): String
}

class Circle(private val radius: Double) : Drawable {
    override fun draw(): String = "Drawing a circle with radius $radius"
}

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView {
            Basic1Theme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    KotlinFundamentalsDemo(

```

```

        modifier = Modifier.padding(innerPadding)
    )
}
}
}
}
}

```

@Composable

```

fun KotlinFundamentalsDemo(modifier: Modifier = Modifier) {
    var result by remember { mutableStateOf("") }

```

```

    Column(
        modifier = modifier
        .fillMaxSize()
        .verticalScroll(rememberScrollState())
        .padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        Text(
            text = "Kotlin Fundamentals Demo",
            fontSize = 24.sp,
            fontWeight = FontWeight.Bold
        )
    }

```

// Variables and Data Types

```

SectionCard(title = "Variables & Data Types") {
    demonstrateVariablesAndDataTypes()
}

```

// Operators

```

SectionCard(title = "Operators") {
    demonstrateOperators()
}

```

// Control Statements

```

SectionCard(title = "Control Statements") {
    demonstrateControlStatements()
}

```

// Null Safety

```

SectionCard(title = "Null Safety") {
    demonstrateNullSafety()
}

```

// Functions

```

SectionCard(title = "Functions") {
    demonstrateFunctions()
}

```

```

    }

    // Classes and Objects
    SectionCard(title = "Classes & Objects") {
        demonstrateClassesAndObjects()
    }

    // Interactive Example
    SectionCard(title = "Interactive Example") {
        InteractiveExample { newResult ->
            result = newResult
        }
    }

    if (result.isNotEmpty()) {
        Card(
            modifier = Modifier.fillMaxWidth(),
            colors = CardDefaults.cardColors(
                containerColor = MaterialTheme.colorScheme.primaryContainer
            )
        ) {
            Text(
                text = "Result: $result",
                modifier = Modifier.padding(16.dp),
                fontSize = 16.sp
            )
        }
    }
}

```

```

@Composable
fun SectionCard(title: String, content: @Composable () -> Unit) {
    Card(
        modifier = Modifier.fillMaxWidth(),
        elevation = CardDefaults.cardElevation(defaultElevation = 4.dp)
    ) {
        Column(
            modifier = Modifier.padding(16.dp),
            verticalArrangement = Arrangement.spacedBy(8.dp)
        ) {
            Text(
                text = title,
                fontSize = 18.sp,
                fontWeight = FontWeight.Bold,
                color = MaterialTheme.colorScheme.primary
            )
            content()
        }
    }
}

```

```
}  
}  
}
```

@Composable

```
fun demonstrateVariablesAndDataTypes() {
```

```
    // Immutable variable (val)
```

```
    val immutableName: String = "Kotlin"
```

```
    // Mutable variable (var)
```

```
    var mutableAge: Int = 10
```

```
    mutableAge += 1
```

```
    // Type inference
```

```
    val inferredDouble = 3.14159
```

```
    // Different data types
```

```
    val byteValue: Byte = 127
```

```
    val shortValue: Short = 32767
```

```
    val intValue: Int = 2147483647
```

```
    val longValue: Long = 9223372036854775807L
```

```
    val floatValue: Float = 3.14f
```

```
    val doubleValue: Double = 2.71828
```

```
    val charValue: Char = 'K'
```

```
    val booleanValue: Boolean = true
```

```
    val stringValue: String = "Hello Kotlin"
```

```
    val arrayValue: IntArray = intArrayOf(1, 2, 3, 4, 5)
```

```
    Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
```

```
        Text("val (immutable): $immutableName")
```

```
        Text("var (mutable): $mutableAge")
```

```
        Text("Type inference: $inferredDouble")
```

```
        Text("Byte: $byteValue")
```

```
        Text("Int: $intValue")
```

```
        Text("Long: $longValue")
```

```
        Text("Float: $floatValue")
```

```
        Text("Double: $doubleValue")
```

```
        Text("Char: $charValue")
```

```
        Text("Boolean: $booleanValue")
```

```
        Text("String: $stringValue")
```

```
        Text("Array: ${arrayValue.contentToString()}")
```

```
    }
```

```
}
```

@Composable

```
fun demonstrateOperators() {
```

```
    val a = 10
```

```
    val b = 3
```

```
// Arithmetic operators
```

```
val sum = a + b
```

```
val difference = a - b
```

```
val product = a * b
```

```
val quotient = a / b
```

```
val remainder = a % b
```

```
// Comparison operators
```

```
val isEqual = a == b
```

```
val isNotEqual = a != b
```

```
val isGreater = a > b
```

```
val isLess = a < b
```

```
// Logical operators
```

```
val x = true
```

```
val y = false
```

```
val andResult = x && y
```

```
val orResult = x || y
```

```
val notResult = !x
```

```
Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
```

```
    Text("Arithmetic Operators (a=$a, b=$b):")
```

```
    Text(" a + b = $sum")
```

```
    Text(" a - b = $difference")
```

```
    Text(" a * b = $product")
```

```
    Text(" a / b = $quotient")
```

```
    Text(" a % b = $remainder")
```

```
    Text("Comparison Operators:")
```

```
    Text(" a == b = $isEqual")
```

```
    Text(" a != b = $isNotEqual")
```

```
    Text(" a > b = $isGreater")
```

```
    Text(" a < b = $isLess")
```

```
    Text("Logical Operators (x=$x, y=$y):")
```

```
    Text(" x && y = $andResult")
```

```
    Text(" x || y = $orResult")
```

```
    Text(" !x = $notResult")
```

```
}
```

```
}
```

```
@Composable
```

```
fun demonstrateControlStatements() {
```

```
    val score = 85
```

```
    val grade = when {
```

```
        score >= 90 -> "A"
```

```
        score >= 80 -> "B"
```

```
    score >= 70 -> "C"
    score >= 60 -> "D"
    else -> "F"
}
```

```
val numbers = listOf(1, 2, 3, 4, 5)
val evenNumbers = mutableListOf<Int>()
```

```
// For loop
for (number in numbers) {
    if (number % 2 == 0) {
        evenNumbers.add(number)
    }
}
```

```
// While loop simulation
var countdown = 5
val countdownResult = buildString {
    while (countdown > 0) {
        append("$countdown ")
        countdown--
    }
    append("Blast off!")
}
```

```
Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
    Text("If/When Statement:")
    Text(" Score: $score -> Grade: $grade")

    Text("For Loop:")
    Text(" Numbers: ${numbers.joinToString()}")
    Text(" Even numbers: ${evenNumbers.joinToString()}")

    Text("While Loop:")
    Text(" Countdown: $countdownResult")
}
}
```

```
@Composable
fun demonstrateNullSafety() {
    // Nullable type
    val nullableString: String? = "Hello"
    val nullString: String? = null

    // Safe call operator
    val length1 = nullableString?.length

    // Elvis operator
```

```
val length2 = nullString?.length ?: 0
```

```
// Non-null asserted call (use with caution)
```

```
val length3 = nullableString!!.length
```

```
// Safe cast
```

```
val anyValue: Any = "Kotlin"
```

```
val safeCast = anyValue as? String
```

```
// Let scope function
```

```
val result = nullableString?.let {
```

```
    "Length of '$it' is ${it.length}"
```

```
}
```

```
Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
```

```
    Text("Nullable String: $nullableString")
```

```
    Text("Null String: $nullString")
```

```
    Text("Safe call (?): $length1")
```

```
    Text("Elvis (?): $length2")
```

```
    Text("Non-null asserted (!): $length3")
```

```
    Text("Safe cast (as?): $safeCast")
```

```
    Text("Let scope function: $result")
```

```
}
```

```
}
```

```
@Composable
```

```
fun demonstrateFunctions() {
```

```
    // Lambda function
```

```
    val multiply = { a: Int, b: Int -> a * b }
```

```
    // Extension function
```

```
    fun String.isPalindrome(): Boolean {
```

```
        val cleaned = this.lowercase().filter { it.isLetter() }
```

```
        return cleaned == cleaned.reversed()
```

```
}
```

```
val sum = add(5, 3)
```

```
val greeting = greet("Kotlin")
```

```
val sumVarargs = sumAll(1, 2, 3, 4, 5)
```

```
val product = operate(4, 5, multiply)
```

```
val palindrome = "Racecar".isPalindrome()
```

```
val fact = factorial(5)
```

```
Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
```

```
    Text("Basic Function: add(5, 3) = $sum")
```

```
    Text("Default Parameter: greet(\"Kotlin\") = \"$greeting\"")
```

```
    Text("Varargs: sumAll(1,2,3,4,5) = $sumVarargs")
```

```
    Text("Higher-order: operate(4,5,multiply) = $product")
```

```

        Text("Extension: \"Racecar\".isPalindrome() = $palindrome")
        Text("Tail-recursive: factorial(5) = $fact")
    }
}

```

@Composable

```
fun demonstrateClassesAndObjects() {
```

```
    // Usage examples
```

```
    val person = Person("Alice", 25)
```

```
    val user = User(1, "john_doe", "john@example.com")
```

```
    val priority = Priority.HIGH
```

```
    val config = AppConfig.getConfig()
```

```
    val area = MathUtils.circleArea(5.0)
```

```
    val dog = Dog("Buddy")
```

```
    val circle = Circle(10.0)
```

```
    Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
```

```
        Text("Class: ${person.introduce()}")
```

```
        Text("Data class: User(username=${user.username}, email=${user.email})")
```

```
        Text("Enum: ${priority.getDisplayName()} = ${priority.value}")
```

```
        Text("Object: $config")
```

```
        Text("Companion object: Circle area (r=5) = $area")
```

```
        Text("Abstract class: ${dog.makeSound()}")
```

```
        Text("Interface: ${circle.draw()}")
```

```
    }
```

```
}
```

@Composable

```
fun InteractiveExample(onResult: (String) -> Unit) {
```

```
    var input1 by remember { mutableStateOf("") }

```

```
    var input2 by remember { mutableStateOf("") }

```

```
    var operation by remember { mutableStateOf("add") }
```

```
    Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
```

```
        Text("Try it yourself!")
```

```
        OutlinedTextField(

```

```
            value = input1,
```

```
            onChange = { input1 = it },
```

```
            label = { Text("Number 1") },
```

```
            modifier = Modifier.fillMaxWidth()

```

```
        )
```

```
        OutlinedTextField(

```

```
            value = input2,
```

```
            onChange = { input2 = it },
```

```
            label = { Text("Number 2") },
```

```
            modifier = Modifier.fillMaxWidth()

```

```

    )

    Row(
        horizontalArrangement = Arrangement.spacedBy(8.dp),
        modifier = Modifier.fillMaxWidth()
    ) {
        RadioButton(
            selected = operation == "add",
            onClick = { operation = "add" }
        )
        Text("Add")

        RadioButton(
            selected = operation == "subtract",
            onClick = { operation = "subtract" }
        )
        Text("Subtract")

        RadioButton(
            selected = operation == "multiply",
            onClick = { operation = "multiply" }
        )
        Text("Multiply")
    }

    Button(
        onClick = {
            val num1 = input1.toIntOrNull() ?: 0
            val num2 = input2.toIntOrNull() ?: 0
            val result = when (operation) {
                "add" -> num1 + num2
                "subtract" -> num1 - num2
                "multiply" -> num1 * num2
                else -> 0
            }
            onResult("$num1 $operation $num2 = $result")
        },
        modifier = Modifier.align(Alignment.CenterHorizontally)
    ) {
        Text("Calculate")
    }
}

```

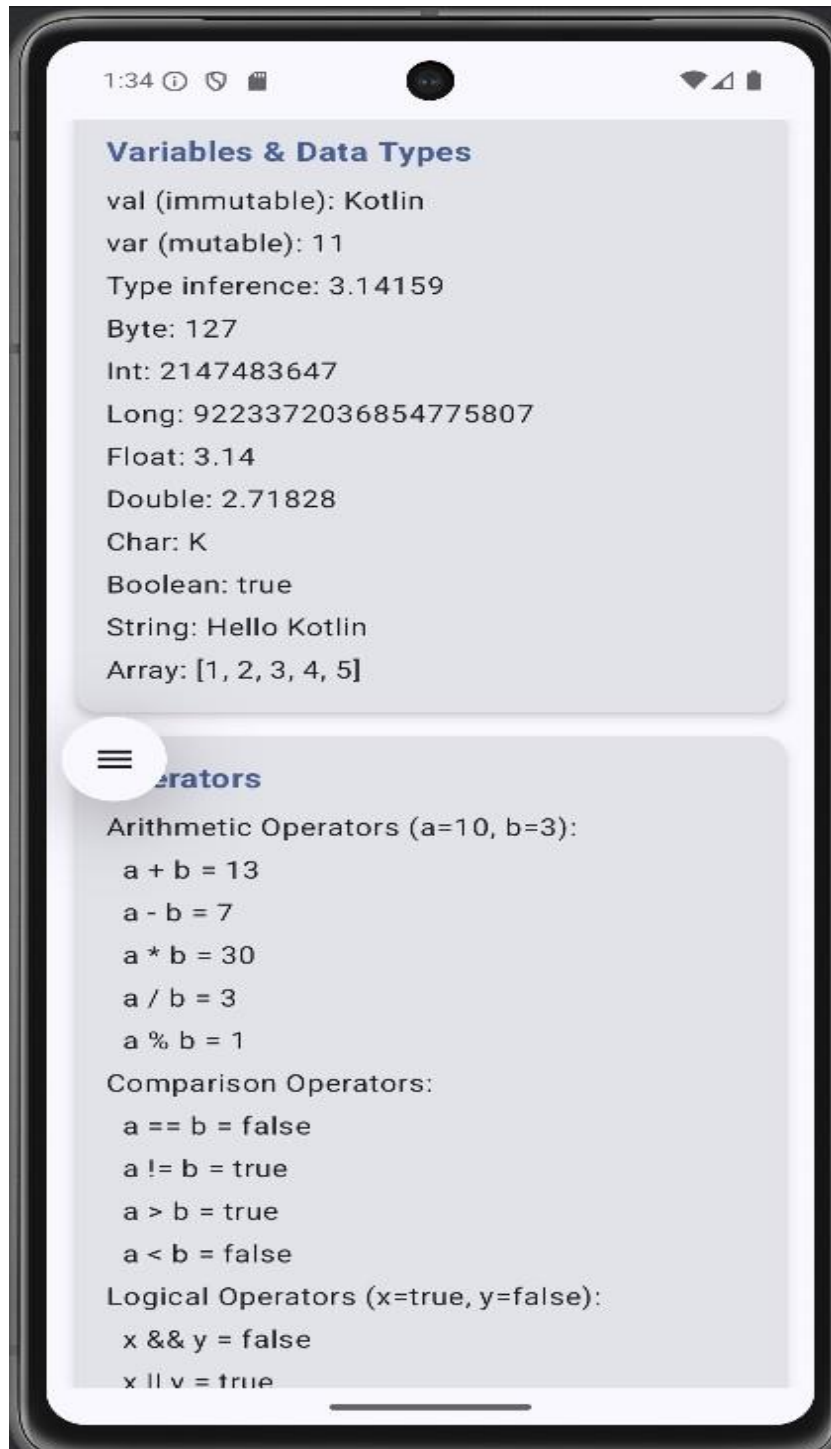
```

@Preview(showBackground = true)
@Composable
fun KotlinFundamentalsDemoPreview() {
    Basic1Theme {

```

```
        KotlinFundamentalsDemo()  
    }  
}
```

Output



Expt No : 2

Date:

Functions

Aim

To understand and implement different types of functions in Kotlin.

Algorithm

1. Create a Kotlin file.
2. Define a function using fun keyword.
3. Pass parameters to the function.
4. Specify return type.
5. Call the function from main() or activity.
6. Implement default arguments.
7. Execute the program.

Coding

```
package com.example.basic1
```

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.compose.ui.tooling.preview.Preview
import com.example.basic1.ui.theme.Basic1Theme

// Function examples defined outside composable
fun add(a: Int, b: Int): Int = a + b
fun greet(name: String, greeting: String = "Hello"): String = "$greeting, $name!"
fun sumAll(vararg numbers: Int): Int = numbers.sum()
fun operate(a: Int, b: Int, operation: (Int, Int) -> Int): Int = operation(a, b)
tailrec fun factorial(n: Int, acc: Int = 1): Int = if (n <= 1) acc else factorial(n - 1, acc * n)

// Class examples defined outside composable
class Person(val name: String, var age: Int) {
```

```

    fun introduce(): String = "Hi, I'm $name and I'm $age years old"
}

data class User(val id: Int, val username: String, val email: String)

enum class Priority(val value: Int) {
    LOW(1), MEDIUM(2), HIGH(3), URGENT(4);
    fun getDisplayName(): String = name.lowercase().replaceFirstChar { it.uppercase() }
}

object AppConfig {
    const val APP_NAME = "Kotlin Demo"
    const val VERSION = "1.0.0"
    fun getConfig(): String = "$APP_NAME v$VERSION"
}

class MathUtils {
    companion object {
        const val PI = 3.14159
        fun circleArea(radius: Double): Double = PI * radius * radius
    }
}

abstract class Animal(val name: String) {
    abstract fun makeSound(): String
    fun sleep(): String = "$name is sleeping"
}

class Dog(name: String) : Animal(name) {
    override fun makeSound(): String = "$name says: Woof!"
}

interface Drawable {
    fun draw(): String
}

class Circle(private val radius: Double) : Drawable {
    override fun draw(): String = "Drawing a circle with radius $radius"
}

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView {
            Basic1Theme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    KotlinFundamentalsDemo(

```

```

        modifier = Modifier.padding(innerPadding)
    )
}
}
}
}
}

```

@Composable

```

fun KotlinFundamentalsDemo(modifier: Modifier = Modifier) {
    var result by remember { mutableStateOf("") }

```

```

    Column(
        modifier = modifier
        .fillMaxSize()
        .verticalScroll(rememberScrollState())
        .padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        Text(
            text = "Kotlin Fundamentals Demo",
            fontSize = 24.sp,
            fontWeight = FontWeight.Bold
        )
    }

```

// Variables and Data Types

```

SectionCard(title = "Variables & Data Types") {
    demonstrateVariablesAndDataTypes()
}

```

// Operators

```

SectionCard(title = "Operators") {
    demonstrateOperators()
}

```

// Control Statements

```

SectionCard(title = "Control Statements") {
    demonstrateControlStatements()
}

```

// Null Safety

```

SectionCard(title = "Null Safety") {
    demonstrateNullSafety()
}

```

// Functions

```

SectionCard(title = "Functions") {
    demonstrateFunctions()
}

```

```

    }

    // Classes and Objects
    SectionCard(title = "Classes & Objects") {
        demonstrateClassesAndObjects()
    }

    // Interactive Example
    SectionCard(title = "Interactive Example") {
        InteractiveExample { newResult ->
            result = newResult
        }
    }

    if (result.isNotEmpty()) {
        Card(
            modifier = Modifier.fillMaxWidth(),
            colors = CardDefaults.cardColors(
                containerColor = MaterialTheme.colorScheme.primaryContainer
            )
        ) {
            Text(
                text = "Result: $result",
                modifier = Modifier.padding(16.dp),
                fontSize = 16.sp
            )
        }
    }
}

```

```

@Composable
fun SectionCard(title: String, content: @Composable () -> Unit) {
    Card(
        modifier = Modifier.fillMaxWidth(),
        elevation = CardDefaults.cardElevation(defaultElevation = 4.dp)
    ) {
        Column(
            modifier = Modifier.padding(16.dp),
            verticalArrangement = Arrangement.spacedBy(8.dp)
        ) {
            Text(
                text = title,
                fontSize = 18.sp,
                fontWeight = FontWeight.Bold,
                color = MaterialTheme.colorScheme.primary
            )
            content()
        }
    }
}

```

```
}  
}  
}
```

@Composable

```
fun demonstrateVariablesAndDataTypes() {
```

```
    // Immutable variable (val)
```

```
    val immutableName: String = "Kotlin"
```

```
    // Mutable variable (var)
```

```
    var mutableAge: Int = 10
```

```
    mutableAge += 1
```

```
    // Type inference
```

```
    val inferredDouble = 3.14159
```

```
    // Different data types
```

```
    val byteValue: Byte = 127
```

```
    val shortValue: Short = 32767
```

```
    val intValue: Int = 2147483647
```

```
    val longValue: Long = 9223372036854775807L
```

```
    val floatValue: Float = 3.14f
```

```
    val doubleValue: Double = 2.71828
```

```
    val charValue: Char = 'K'
```

```
    val booleanValue: Boolean = true
```

```
    val stringValue: String = "Hello Kotlin"
```

```
    val arrayValue: IntArray = intArrayOf(1, 2, 3, 4, 5)
```

```
    Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
```

```
        Text("val (immutable): $immutableName")
```

```
        Text("var (mutable): $mutableAge")
```

```
        Text("Type inference: $inferredDouble")
```

```
        Text("Byte: $byteValue")
```

```
        Text("Int: $intValue")
```

```
        Text("Long: $longValue")
```

```
        Text("Float: $floatValue")
```

```
        Text("Double: $doubleValue")
```

```
        Text("Char: $charValue")
```

```
        Text("Boolean: $booleanValue")
```

```
        Text("String: $stringValue")
```

```
        Text("Array: ${arrayValue.contentToString()}")
```

```
    }
```

```
}
```

@Composable

```
fun demonstrateOperators() {
```

```
    val a = 10
```

```
    val b = 3
```

```
// Arithmetic operators
```

```
val sum = a + b
```

```
val difference = a - b
```

```
val product = a * b
```

```
val quotient = a / b
```

```
val remainder = a % b
```

```
// Comparison operators
```

```
val isEqual = a == b
```

```
val isNotEqual = a != b
```

```
val isGreater = a > b
```

```
val isLess = a < b
```

```
// Logical operators
```

```
val x = true
```

```
val y = false
```

```
val andResult = x && y
```

```
val orResult = x || y
```

```
val notResult = !x
```

```
Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
```

```
    Text("Arithmetic Operators (a=$a, b=$b):")
```

```
    Text(" a + b = $sum")
```

```
    Text(" a - b = $difference")
```

```
    Text(" a * b = $product")
```

```
    Text(" a / b = $quotient")
```

```
    Text(" a % b = $remainder")
```

```
    Text("Comparison Operators:")
```

```
    Text(" a == b = $isEqual")
```

```
    Text(" a != b = $isNotEqual")
```

```
    Text(" a > b = $isGreater")
```

```
    Text(" a < b = $isLess")
```

```
    Text("Logical Operators (x=$x, y=$y):")
```

```
    Text(" x && y = $andResult")
```

```
    Text(" x || y = $orResult")
```

```
    Text(" !x = $notResult")
```

```
}
```

```
}
```

```
@Composable
```

```
fun demonstrateControlStatements() {
```

```
    val score = 85
```

```
    val grade = when {
```

```
        score >= 90 -> "A"
```

```
        score >= 80 -> "B"
```

```
    score >= 70 -> "C"
    score >= 60 -> "D"
    else -> "F"
}
```

```
val numbers = listOf(1, 2, 3, 4, 5)
val evenNumbers = mutableListOf<Int>()
```

```
// For loop
for (number in numbers) {
    if (number % 2 == 0) {
        evenNumbers.add(number)
    }
}
```

```
// While loop simulation
var countdown = 5
val countdownResult = buildString {
    while (countdown > 0) {
        append("$countdown ")
        countdown--
    }
    append("Blast off!")
}
```

```
Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
    Text("If/When Statement:")
    Text(" Score: $score -> Grade: $grade")

    Text("For Loop:")
    Text(" Numbers: ${numbers.joinToString()}")
    Text(" Even numbers: ${evenNumbers.joinToString()}")

    Text("While Loop:")
    Text(" Countdown: $countdownResult")
}
}
```

```
@Composable
fun demonstrateNullSafety() {
    // Nullable type
    val nullableString: String? = "Hello"
    val nullString: String? = null

    // Safe call operator
    val length1 = nullableString?.length

    // Elvis operator
```

```
val length2 = nullString?.length ?: 0
```

```
// Non-null asserted call (use with caution)
```

```
val length3 = nullableString!!.length
```

```
// Safe cast
```

```
val anyValue: Any = "Kotlin"
```

```
val safeCast = anyValue as? String
```

```
// Let scope function
```

```
val result = nullableString?.let {  
    "Length of '$it' is ${it.length}"  
}
```

```
Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
```

```
    Text("Nullable String: $nullableString")
```

```
    Text("Null String: $nullString")
```

```
    Text("Safe call (?): $length1")
```

```
    Text("Elvis (?:): $length2")
```

```
    Text("Non-null asserted (!): $length3")
```

```
    Text("Safe cast (as?): $safeCast")
```

```
    Text("Let scope function: $result")
```

```
}
```

```
}
```

```
@Composable
```

```
fun demonstrateFunctions() {
```

```
    // Lambda function
```

```
    val multiply = { a: Int, b: Int -> a * b }
```

```
    // Extension function
```

```
    fun String.isPalindrome(): Boolean {
```

```
        val cleaned = this.lowercase().filter { it.isLetter() }
```

```
        return cleaned == cleaned.reversed()
```

```
}
```

```
val sum = add(5, 3)
```

```
val greeting = greet("Kotlin")
```

```
val sumVarargs = sumAll(1, 2, 3, 4, 5)
```

```
val product = operate(4, 5, multiply)
```

```
val palindrome = "Racecar".isPalindrome()
```

```
val fact = factorial(5)
```

```
Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
```

```
    Text("Basic Function: add(5, 3) = $sum")
```

```
    Text("Default Parameter: greet(\"Kotlin\") = \"$greeting\"")
```

```
    Text("Varargs: sumAll(1,2,3,4,5) = $sumVarargs")
```

```
    Text("Higher-order: operate(4,5,multiply) = $product")
```

```

        Text("Extension: \"Racecar\".isPalindrome() = $palindrome")
        Text("Tail-recursive: factorial(5) = $fact")
    }
}

```

@Composable

```

fun demonstrateClassesAndObjects() {

```

```

    // Usage examples

```

```

    val person = Person("Alice", 25)

```

```

    val user = User(1, "john_doe", "john@example.com")

```

```

    val priority = Priority.HIGH

```

```

    val config = AppConfig.getConfig()

```

```

    val area = MathUtils.circleArea(5.0)

```

```

    val dog = Dog("Buddy")

```

```

    val circle = Circle(10.0)

```

```

    Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {

```

```

        Text("Class: ${person.introduce()}")

```

```

        Text("Data class: User(username=${user.username}, email=${user.email})")

```

```

        Text("Enum: ${priority.getDisplayName()} = ${priority.value}")

```

```

        Text("Object: $config")

```

```

        Text("Companion object: Circle area (r=5) = $area")

```

```

        Text("Abstract class: ${dog.makeSound()}")

```

```

        Text("Interface: ${circle.draw()}")

```

```

    }

```

```

}

```

@Composable

```

fun InteractiveExample(onResult: (String) -> Unit) {

```

```

    var input1 by remember { mutableStateOf("") }

```

```

    var input2 by remember { mutableStateOf("") }

```

```

    var operation by remember { mutableStateOf("add") }

```

```

    Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {

```

```

        Text("Try it yourself!")

```

```

        OutlinedTextField(

```

```

            value = input1,

```

```

            onChange = { input1 = it },

```

```

            label = { Text("Number 1") },

```

```

            modifier = Modifier.fillMaxWidth()

```

```

        )

```

```

        OutlinedTextField(

```

```

            value = input2,

```

```

            onChange = { input2 = it },

```

```

            label = { Text("Number 2") },

```

```

            modifier = Modifier.fillMaxWidth()

```

```

    )

    Row(
        horizontalArrangement = Arrangement.spacedBy(8.dp),
        modifier = Modifier.fillMaxWidth()
    ) {
        RadioButton(
            selected = operation == "add",
            onClick = { operation = "add" }
        )
        Text("Add")

        RadioButton(
            selected = operation == "subtract",
            onClick = { operation = "subtract" }
        )
        Text("Subtract")

        RadioButton(
            selected = operation == "multiply",
            onClick = { operation = "multiply" }
        )
        Text("Multiply")
    }

    Button(
        onClick = {
            val num1 = input1.toIntOrNull() ?: 0
            val num2 = input2.toIntOrNull() ?: 0
            val result = when (operation) {
                "add" -> num1 + num2
                "subtract" -> num1 - num2
                "multiply" -> num1 * num2
                else -> 0
            }
            onResult("$num1 $operation $num2 = $result")
        },
        modifier = Modifier.align(Alignment.CenterHorizontally)
    ) {
        Text("Calculate")
    }
}

```

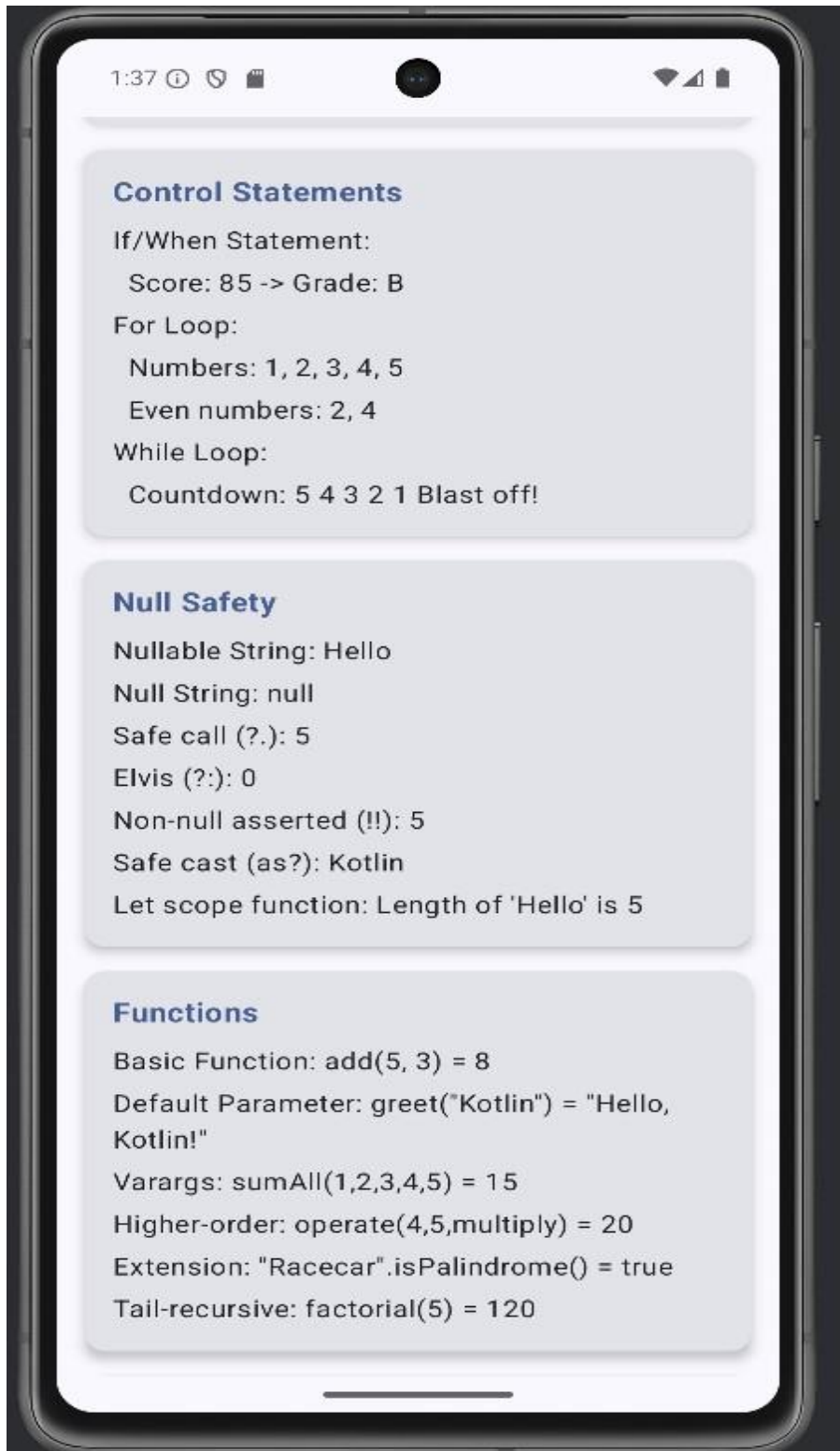
```

@Preview(showBackground = true)
@Composable
fun KotlinFundamentalsDemoPreview() {
    Basic1Theme {

```

```
KotlinFundamentalsDemo()  
}  
}
```

Output



Expt No : 3

Date:

Classes and Objects

Aim

To implement object-oriented programming concepts using classes and objects in Kotlin.

Algorithm

1. Define a class using class keyword.
2. Declare properties inside the class.
3. Define member functions.
4. Create object using constructor.
5. Access class members.
6. Execute and verify output.

Coding

```
package com.example.basic1
```

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.compose.ui.tooling.preview.Preview
import com.example.basic1.ui.theme.Basic1Theme

// Function examples defined outside composable
fun add(a: Int, b: Int): Int = a + b
fun greet(name: String, greeting: String = "Hello"): String = "$greeting, $name!"
fun sumAll(vararg numbers: Int): Int = numbers.sum()
fun operate(a: Int, b: Int, operation: (Int, Int) -> Int): Int = operation(a, b)
tailrec fun factorial(n: Int, acc: Int = 1): Int = if (n <= 1) acc else factorial(n - 1, acc * n)

// Class examples defined outside composable
class Person(val name: String, var age: Int) {
    fun introduce(): String = "Hi, I'm $name and I'm $age years old"
}
```

```

data class User(val id: Int, val username: String, val email: String)

enum class Priority(val value: Int) {
    LOW(1), MEDIUM(2), HIGH(3), URGENT(4);
    fun getDisplayName(): String = name.lowercase().replaceFirstChar { it.uppercase() }
}

object AppConfig {
    const val APP_NAME = "Kotlin Demo"
    const val VERSION = "1.0.0"
    fun getConfig(): String = "$APP_NAME v$VERSION"
}

class MathUtils {
    companion object {
        const val PI = 3.14159
        fun circleArea(radius: Double): Double = PI * radius * radius
    }
}

abstract class Animal(val name: String) {
    abstract fun makeSound(): String
    fun sleep(): String = "$name is sleeping"
}

class Dog(name: String) : Animal(name) {
    override fun makeSound(): String = "$name says: Woof!"
}

interface Drawable {
    fun draw(): String
}

class Circle(private val radius: Double) : Drawable {
    override fun draw(): String = "Drawing a circle with radius $radius"
}

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView {
            Basic1Theme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    KotlinFundamentalsDemo(
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
}

```

```

    }
  }
}

```

@Composable

```

fun KotlinFundamentalsDemo(modifier: Modifier = Modifier) {
    var result by remember { mutableStateOf("") }

```

```

    Column(
        modifier = modifier
            .fillMaxSize()
            .verticalScroll(rememberScrollState())
            .padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        Text(
            text = "Kotlin Fundamentals Demo",
            fontSize = 24.sp,
            fontWeight = FontWeight.Bold
        )
    }

```

// Variables and Data Types

```

SectionCard(title = "Variables & Data Types") {
    demonstrateVariablesAndDataTypes()
}

```

// Operators

```

SectionCard(title = "Operators") {
    demonstrateOperators()
}

```

// Control Statements

```

SectionCard(title = "Control Statements") {
    demonstrateControlStatements()
}

```

// Null Safety

```

SectionCard(title = "Null Safety") {
    demonstrateNullSafety()
}

```

// Functions

```

SectionCard(title = "Functions") {
    demonstrateFunctions()
}

```

```
// Classes and Objects
```

```
SectionCard(title = "Classes & Objects") {  
    demonstrateClassesAndObjects()  
}
```

```
// Interactive Example
```

```
SectionCard(title = "Interactive Example") {  
    InteractiveExample { newResult ->  
        result = newResult  
    }  
}
```

```
if (result.isNotEmpty()) {  
    Card(  
        modifier = Modifier.fillMaxWidth(),  
        colors = CardDefaults.cardColors(  
            containerColor = MaterialTheme.colorScheme.primaryContainer  
        )  
    ) {  
        Text(  
            text = "Result: $result",  
            modifier = Modifier.padding(16.dp),  
            fontSize = 16.sp  
        )  
    }  
}
```

```
@Composable
```

```
fun SectionCard(title: String, content: @Composable () -> Unit) {  
    Card(  
        modifier = Modifier.fillMaxWidth(),  
        elevation = CardDefaults.cardElevation(defaultElevation = 4.dp)  
    ) {  
        Column(  
            modifier = Modifier.padding(16.dp),  
            verticalArrangement = Arrangement.spacedBy(8.dp)  
        ) {  
            Text(  
                text = title,  
                fontSize = 18.sp,  
                fontWeight = FontWeight.Bold,  
                color = MaterialTheme.colorScheme.primary  
            )  
            content()  
        }  
    }  
}
```

```
}
```

```
@Composable
```

```
fun demonstrateVariablesAndDataTypes() {
```

```
    // Immutable variable (val)
```

```
    val immutableName: String = "Kotlin"
```

```
    // Mutable variable (var)
```

```
    var mutableAge: Int = 10
```

```
    mutableAge += 1
```

```
    // Type inference
```

```
    val inferredDouble = 3.14159
```

```
    // Different data types
```

```
    val byteValue: Byte = 127
```

```
    val shortValue: Short = 32767
```

```
    val intValue: Int = 2147483647
```

```
    val longValue: Long = 9223372036854775807L
```

```
    val floatValue: Float = 3.14f
```

```
    val doubleValue: Double = 2.71828
```

```
    val charValue: Char = 'K'
```

```
    val booleanValue: Boolean = true
```

```
    val stringValue: String = "Hello Kotlin"
```

```
    val arrayValue: IntArray = intArrayOf(1, 2, 3, 4, 5)
```

```
    Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
```

```
        Text("val (immutable): $immutableName")
```

```
        Text("var (mutable): $mutableAge")
```

```
        Text("Type inference: $inferredDouble")
```

```
        Text("Byte: $byteValue")
```

```
        Text("Int: $intValue")
```

```
        Text("Long: $longValue")
```

```
        Text("Float: $floatValue")
```

```
        Text("Double: $doubleValue")
```

```
        Text("Char: $charValue")
```

```
        Text("Boolean: $booleanValue")
```

```
        Text("String: $stringValue")
```

```
        Text("Array: ${arrayValue.contentToString()}")
```

```
    }
```

```
}
```

```
@Composable
```

```
fun demonstrateOperators() {
```

```
    val a = 10
```

```
    val b = 3
```

```
    // Arithmetic operators
```

```
val sum = a + b
val difference = a - b
val product = a * b
val quotient = a / b
val remainder = a % b
```

```
// Comparison operators
```

```
val isEqual = a == b
val isNotEqual = a != b
val isGreater = a > b
val isLess = a < b
```

```
// Logical operators
```

```
val x = true
val y = false
val andResult = x && y
val orResult = x || y
val notResult = !x
```

```
Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
```

```
    Text("Arithmetic Operators (a=$a, b=$b):")
```

```
    Text(" a + b = $sum")
```

```
    Text(" a - b = $difference")
```

```
    Text(" a * b = $product")
```

```
    Text(" a / b = $quotient")
```

```
    Text(" a % b = $remainder")
```

```
    Text("Comparison Operators:")
```

```
    Text(" a == b = $isEqual")
```

```
    Text(" a != b = $isNotEqual")
```

```
    Text(" a > b = $isGreater")
```

```
    Text(" a < b = $isLess")
```

```
    Text("Logical Operators (x=$x, y=$y):")
```

```
    Text(" x && y = $andResult")
```

```
    Text(" x || y = $orResult")
```

```
    Text(" !x = $notResult")
```

```
}
```

```
}
```

```
@Composable
```

```
fun demonstrateControlStatements() {
```

```
    val score = 85
```

```
    val grade = when {
```

```
        score >= 90 -> "A"
```

```
        score >= 80 -> "B"
```

```
        score >= 70 -> "C"
```

```
        score >= 60 -> "D"
```

```

    else -> "F"
}

val numbers = listOf(1, 2, 3, 4, 5)
val evenNumbers = mutableListOf<Int>()

```

```

// For loop
for (number in numbers) {
    if (number % 2 == 0) {
        evenNumbers.add(number)
    }
}

```

```

// While loop simulation
var countdown = 5
val countdownResult = buildString {
    while (countdown > 0) {
        append("$countdown ")
        countdown--
    }
    append("Blast off!")
}

```

```

Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
    Text("If/When Statement:")
    Text(" Score: $score -> Grade: $grade")

    Text("For Loop:")
    Text(" Numbers: ${numbers.joinToString()}")
    Text(" Even numbers: ${evenNumbers.joinToString()}")

    Text("While Loop:")
    Text(" Countdown: $countdownResult")
}
}

```

```

@Composable
fun demonstrateNullSafety() {
    // Nullable type
    val nullableString: String? = "Hello"
    val nullString: String? = null

    // Safe call operator
    val length1 = nullableString?.length

    // Elvis operator
    val length2 = nullString?.length ?: 0
}

```

```
// Non-null asserted call (use with caution)
val length3 = nullableString!!.length
```

```
// Safe cast
val anyValue: Any = "Kotlin"
val safeCast = anyValue as? String
```

```
// Let scope function
val result = nullableString?.let {
    "Length of '$it' is ${it.length}"
}
```

```
Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
    Text("Nullable String: $nullableString")
    Text("Null String: $nullString")
    Text("Safe call (?.): $length1")
    Text("Elvis (?:): $length2")
    Text("Non-null asserted (!!): $length3")
    Text("Safe cast (as?): $safeCast")
    Text("Let scope function: $result")
}
}
```

```
@Composable
fun demonstrateFunctions() {
    // Lambda function
    val multiply = { a: Int, b: Int -> a * b }

    // Extension function
    fun String.isPalindrome(): Boolean {
        val cleaned = this.lowercase().filter { it.isLetter() }
        return cleaned == cleaned.reversed()
    }
}
```

```
val sum = add(5, 3)
val greeting = greet("Kotlin")
val sumVarargs = sumAll(1, 2, 3, 4, 5)
val product = operate(4, 5, multiply)
val palindrome = "Racecar".isPalindrome()
val fact = factorial(5)
```

```
Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
    Text("Basic Function: add(5, 3) = $sum")
    Text("Default Parameter: greet(\"Kotlin\") = \"$greeting\"")
    Text("Varargs: sumAll(1,2,3,4,5) = $sumVarargs")
    Text("Higher-order: operate(4,5,multiply) = $product")
    Text("Extension: \"Racecar\".isPalindrome() = $palindrome")
    Text("Tail-recursive: factorial(5) = $fact")
}
```

```
}  
}
```

@Composable

```
fun demonstrateClassesAndObjects() {
```

```
    // Usage examples
```

```
    val person = Person("Alice", 25)
```

```
    val user = User(1, "john_doe", "john@example.com")
```

```
    val priority = Priority.HIGH
```

```
    val config = AppConfig.getConfig()
```

```
    val area = MathUtils.circleArea(5.0)
```

```
    val dog = Dog("Buddy")
```

```
    val circle = Circle(10.0)
```

```
    Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
```

```
        Text("Class: ${person.introduce()}")
```

```
        Text("Data class: User(username=${user.username}, email=${user.email})")
```

```
        Text("Enum: ${priority.getDisplayName()} = ${priority.value}")
```

```
        Text("Object: $config")
```

```
        Text("Companion object: Circle area (r=5) = $area")
```

```
        Text("Abstract class: ${dog.makeSound()}")
```

```
        Text("Interface: ${circle.draw()}")
```

```
    }
```

```
}
```

@Composable

```
fun InteractiveExample(onResult: (String) -> Unit) {
```

```
    var input1 by remember { mutableStateOf("") }
```

```
    var input2 by remember { mutableStateOf("") }
```

```
    var operation by remember { mutableStateOf("add") }
```

```
    Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
```

```
        Text("Try it yourself!")
```

```
        OutlinedTextField(
```

```
            value = input1,
```

```
            onChange = { input1 = it },
```

```
            label = { Text("Number 1") },
```

```
            modifier = Modifier.fillMaxWidth()
```

```
        )
```

```
        OutlinedTextField(
```

```
            value = input2,
```

```
            onChange = { input2 = it },
```

```
            label = { Text("Number 2") },
```

```
            modifier = Modifier.fillMaxWidth()
```

```
        )
```

```

Row(
    horizontalArrangement = Arrangement.spacedBy(8.dp),
    modifier = Modifier.fillMaxWidth()
) {
    RadioButton(
        selected = operation == "add",
        onClick = { operation = "add" }
    )
    Text("Add")

    RadioButton(
        selected = operation == "subtract",
        onClick = { operation = "subtract" }
    )
    Text("Subtract")

    RadioButton(
        selected = operation == "multiply",
        onClick = { operation = "multiply" }
    )
    Text("Multiply")
}

Button(
    onClick = {
        val num1 = input1.toIntOrNull() ?: 0
        val num2 = input2.toIntOrNull() ?: 0
        val result = when (operation) {
            "add" -> num1 + num2
            "subtract" -> num1 - num2
            "multiply" -> num1 * num2
            else -> 0
        }
        onResult("$num1 $operation $num2 = $result")
    },
    modifier = Modifier.align(Alignment.CenterHorizontally)
) {
    Text("Calculate")
}
}

@Preview(showBackground = true)
@Composable
fun KotlinFundamentalsDemoPreview() {
    Basic1Theme {
        KotlinFundamentalsDemo()
    }
}

```

}

Output



Expt No : 4

Date:

Build First Android App

Aim

To create and execute a basic Android application using Kotlin.

Algorithm

1. Open Android Studio.
2. Create new project → Empty Activity.
3. Design UI using XML layout.
4. Write Kotlin logic in Activity file.
5. Build the project.
6. Run app on emulator/device.

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:supportRtl="true"
        android:theme="@style/Theme.GraphicalPrimitives"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
```

```

<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity">
<org.rajalakshmi.graphicalprimitives.SampleCanvas
android:layout_width="match_parent"
android:layout_height="match_parent">
</org.rajalakshmi.graphicalprimitives.SampleCanvas>
</androidx.constraintlayout.widget.ConstraintLayout>

```

MainActivity.kt

```

package org.rajalakshmi.graphicalprimitives
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}

```

SampleCanvas.kt

```

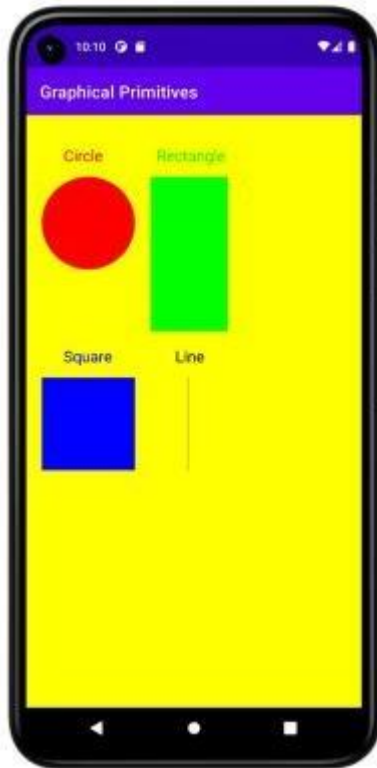
package org.rajalakshmi.graphicalprimitives
import android.content.Context
import android.graphics.Canvas
import android.graphics.Color
import android.graphics.Paint
import android.util.AttributeSet
import android.view.View
class SampleCanvas @JvmOverloads constructor(
    context: Context, attrs: AttributeSet? = null, defStyleAttr: Int = 0
) : View(context, attrs, defStyleAttr) {
    override fun onDraw(canvas: Canvas?) {
        super.onDraw(canvas)
        val paint : Paint = Paint()
        paint.setColor(Color.YELLOW)
        canvas?.drawPaint(paint)
        paint.setTextSize(50f);
        paint.setColor(Color.RED);
        canvas?.drawText("Circle", 120f, 150f, paint);
        canvas?.drawCircle(200f, 350f, 150f, paint);
        paint.setColor(Color.GREEN);

        canvas?.drawText("Rectangle", 420f, 150f, paint);
        canvas?.drawRect(400f, 200f, 650f, 700f, paint);
    }
}

```

```
paint.setColor(Color.BLUE);  
canvas?.drawText("Square", 120f, 800f, paint);  
canvas?.drawRect(50f, 850f, 350f, 1150f, paint);  
paint.setColor(Color.BLACK);  
canvas?.drawText("Line", 480f, 800f, paint);  
canvas?.drawLine(520f, 850f, 520f, 1150f, paint);  
}  
}
```

Output



Expt No : 5

Date:

Layouts

Aim

To design user interfaces using Android layout managers.

Algorithm

1. Open XML layout file.
2. Select layout type.
3. Add UI components.
4. Set layout parameters.
5. Preview and run application.

Mainactivity.kt

```
package com.example.androidlayoutmanagers
```

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.*
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.androidlayoutmanagers.ui.theme.AndroidLayoutManagersTheme
```

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            AndroidLayoutManagersTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    LayoutManagersApp()
                }
            }
        }
    }
}
```

```

    }
  }
}

```

```

@Composable
fun LayoutManagersApp() {
    var selectedDemo by remember { mutableStateOf<DemoType?>(null) }

    selectedDemo?.let { demo ->
        DemoScreen(demo = demo, onBack = { selectedDemo = null })
    } ?: MainScreen(onDemoSelected = { selectedDemo = it })
}

```

```

@Composable
fun MainScreen(onDemoSelected: (DemoType) -> Unit) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .verticalScroll(rememberScrollState())
            .padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        Text(
            text = "Android Layout Managers",
            fontSize = 32.sp,
            fontWeight = FontWeight.Bold,
            modifier = Modifier.align(Alignment.CenterHorizontally)
        )

        Text(
            text = "Explore different layout managers in Jetpack Compose",
            fontSize = 16.sp,
            color = MaterialTheme.colorScheme.onSurfaceVariant,
            modifier = Modifier.align(Alignment.CenterHorizontally)
        )

        Spacer(modifier = Modifier.height(16.dp))

        DemoType.values().forEach { demo ->
            Card(
                modifier = Modifier.fillMaxWidth(),
                elevation = CardDefaults.cardElevation(defaultElevation = 4.dp),
                onClick = { onDemoSelected(demo) }
            ) {
                Column(
                    modifier = Modifier

```

```

        .fillMaxWidth()
        .padding(16.dp)
    ) {
        Text(
            text = demo.title,
            fontSize = 20.sp,
            fontWeight = FontWeight.Bold
        )
        Spacer(modifier = Modifier.height(4.dp))
        Text(
            text = demo.description,
            fontSize = 14.sp,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )
    }
}
}
}
}
}

```

```

@OptIn(ExperimentalMaterial3Api::class)

```

```

@Composable

```

```

fun DemoScreen(demo: DemoType, onBack: () -> Unit) {

```

```

    Column(modifier = Modifier.fillMaxSize()) {

```

```

        TopAppBar(

```

```

            title = { Text(demo.title) },

```

```

            navigationIcon = {

```

```

                IconButton(onClick = onBack) {

```

```

                    Text("← Back")

```

```

                }

```

```

            }

```

```

        )

```

```

        Box(modifier = Modifier.fillMaxSize()) {

```

```

            when (demo) {

```

```

                DemoType.COLUMN -> ColumnExample()

```

```

                DemoType.ROW -> RowExample()

```

```

                DemoType.BOX -> BoxExample()

```

```

                DemoType.LAZY_COLUMN -> LazyColumnExample()

```

```

                DemoType.LAZY_ROW -> LazyRowExample()

```

```

                DemoType.MIXED_LAZY -> MixedLazyLayoutExample()

```

```

                DemoType.CONSTRAINT_BASIC -> ConstraintLayoutBasicExample()

```

```

                DemoType.CONSTRAINT_COMPLEX -> ConstraintLayoutComplexExample()

```

```

                DemoType.CONSTRAINT_GUIDELINES ->

```

```

                ConstraintLayoutGuidelinesExample()

```

```

                DemoType.GRID_FIXED -> LazyVerticalGridExample()

```

```

                DemoType.GRID_ADAPTIVE -> AdaptiveGridExample()

```

```

                DemoType.GRID_VARIABLE -> VariableGridExample()

```

```

        DemoType.PRODUCT_GRID -> ProductGridExample()
    }
}
}

enum class DemoType(
    val title: String,
    val description: String
) {
    COLUMN("Column Layout", "Vertical arrangement of components"),
    ROW("Row Layout", "Horizontal arrangement of components"),
    BOX("Box Layout", "Stacked components with positioning"),
    LAZY_COLUMN("LazyColumn", "Efficient vertical scrolling lists"),
    LAZY_ROW("LazyRow", "Efficient horizontal scrolling lists"),
    MIXED_LAZY("Mixed Lazy Layout", "Combination of LazyColumn and LazyRow"),
    CONSTRAINT_BASIC("ConstraintLayout Basic", "Basic constraint positioning"),
    CONSTRAINT_COMPLEX("ConstraintLayout Complex", "Complex profile layout"),
    CONSTRAINT_GUIDELINES("ConstraintLayout Guidelines", "Layout with
guidelines"),
    GRID_FIXED("Fixed Grid", "Fixed number of columns"),
    GRID_ADAPTIVE("Adaptive Grid", "Adaptive column sizing"),
    GRID_VARIABLE("Variable Grid", "Variable item sizes"),
    PRODUCT_GRID("Product Grid", "Real-world product grid")
}

```

settings.gradle.kts

```

pluginManagement {
    repositories {
        google {
            content {
                includeGroupByRegex("com\\.android.*")
                includeGroupByRegex("com\\.google.*")
                includeGroupByRegex("androidx.*")
            }
        }
        mavenCentral()
        gradlePluginPortal()
    }
}

dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
    }
}

```

```

    }
}

rootProject.name = "Android layout managers"
include(":app")

```

ConstraintsLayoutExamples.kts

```

package com.example.androidlayoutmanagers
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.constraintlayout.compose.ConstraintLayout
import androidx.constraintlayout.compose.ConstraintSet
import androidx.constraintlayout.compose.Dimension

```

```

@Composable
fun ConstraintLayoutBasicExample() {
    ConstraintLayout(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
    ) {
        val (title, subtitle, button, image) = createRefs()

        Text(
            text = "ConstraintLayout Basic",
            fontSize = 24.sp,
            fontWeight = FontWeight.Bold,
            modifier = Modifier.constrainAs(title) {
                top.linkTo(parent.top, margin = 16.dp)

```

```

        start.linkTo(parent.start)
        end.linkTo(parent.end)
    }
)

Text(
    text = "Positioning elements with constraints",
    fontSize = 16.sp,
    color = MaterialTheme.colorScheme.onSurfaceVariant,
    modifier = Modifier.constrainAs(subtitle) {
        top.linkTo(title.bottom, margin = 8.dp)
        start.linkTo(parent.start)
        end.linkTo(parent.end)
    }
)

Button(
    onClick = { },
    modifier = Modifier.constrainAs(button) {
        bottom.linkTo(parent.bottom, margin = 32.dp)
        start.linkTo(parent.start)
        end.linkTo(parent.end)
    }
) {
    Text("Centered Button")
}

Box(
    modifier = Modifier
        .size(100.dp)
        .background(
            Color.Blue,
            RoundedCornerShape(12.dp)
        )
        .constrainAs(image) {
            top.linkTo(subtitle.bottom, margin = 24.dp)
            bottom.linkTo(button.top, margin = 24.dp)
            start.linkTo(parent.start)
            end.linkTo(parent.end)
        },
    contentAlignment = Alignment.Center
) {
    Text(
        text = "Center Box",
        color = Color.White,
        fontWeight = FontWeight.Bold
    )
}

```

```
}  
}
```

@Composable

```
fun ConstraintLayoutComplexExample() {  
    ConstraintLayout(  
        modifier = Modifier  
            .fillMaxSize()  
            .padding(16.dp)  
    ) {  
        val (header, profilePic, userName, userEmail, statsContainer,  
            bioText, actionButton, settingsButton) = createRefs()  
  
        val barrier = createEndBarrier(profilePic, userName, userEmail)  
  
        Text(  
            text = "Profile Layout",  
            fontSize = 20.sp,  
            fontWeight = FontWeight.Bold,  
            modifier = Modifier.constrainAs(header) {  
                top.linkTo(parent.top)  
                start.linkTo(parent.start)  
                end.linkTo(parent.end)  
            }  
        )  
  
        Box(  
            modifier = Modifier  
                .size(80.dp)  
                .background(  
                    Color.Green,  
                    RoundedCornerShape(40.dp)  
                )  
            .constrainAs(profilePic) {  
                top.linkTo(header.bottom, margin = 16.dp)  
                start.linkTo(parent.start)  
            },  
            contentAlignment = Alignment.Center  
        ) {  
            Text(  
                text = "PIC",  
                color = Color.White,  
                fontWeight = FontWeight.Bold  
            )  
        }  
  
        Text(  
            text = "John Doe",
```

```

        fontSize = 18.sp,
        fontWeight = FontWeight.Bold,
        modifier = Modifier.constrainAs(userName) {
            start.linkTo(profilePic.end, margin = 16.dp)
            top.linkTo(profilePic.top)
        }
    )

    Text(
        text = "john.doe@example.com",
        fontSize = 14.sp,
        color = MaterialTheme.colorScheme.onSurfaceVariant,
        modifier = Modifier.constrainAs(userEmail) {
            start.linkTo(profilePic.end, margin = 16.dp)
            top.linkTo(userName.bottom, margin = 4.dp)
        }
    )

```

```

    Box(
        modifier = Modifier
            .constrainAs(statsContainer) {
                top.linkTo(profilePic.bottom, margin = 16.dp)
                start.linkTo(parent.start)
                end.linkTo(parent.end)
                width = Dimension.fillToConstraints
            }
    ) {
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.SpaceEvenly
        ) {
            StatItem("Posts", "150", Modifier)
            StatItem("Followers", "2.5K", Modifier)
            StatItem("Following", "500", Modifier)
        }
    }
}

```

```

    Text(
        text = "Passionate developer and Android enthusiast. Love creating beautiful and functional mobile applications.",
        fontSize = 14.sp,
        modifier = Modifier.constrainAs(bioText) {
            top.linkTo(statsContainer.bottom, margin = 16.dp)
            start.linkTo(parent.start)
            end.linkTo(parent.end)
            width = Dimension.fillToConstraints
        }
    )
}

```

```

Button(
    onClick = { },
    modifier = Modifier.constrainAs(actionButton) {
        top.linkTo(bioText.bottom, margin = 16.dp)
        start.linkTo(parent.start)
        end.linkTo(settingsButton.start, margin = 8.dp)
        width = Dimension.fillToConstraints
    }
) {
    Text("Follow")
}

```

```

OutlinedButton(
    onClick = { },
    modifier = Modifier.constrainAs(settingsButton) {
        top.linkTo(bioText.bottom, margin = 16.dp)
        end.linkTo(parent.end)
        width = Dimension.value(120.dp)
    }
) {
    Text("Message")
}
}

```

```

@Composable
private fun StatItem(label: String, value: String, modifier: Modifier) {
    Column(
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = modifier
    ) {
        Text(
            text = value,
            fontSize = 18.sp,
            fontWeight = FontWeight.Bold
        )
        Text(
            text = label,
            fontSize = 12.sp,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )
    }
}

```

```

@Composable
fun ConstraintLayoutGuidelinesExample() {
    ConstraintLayout(

```

```

        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
    ) {
        val (title, leftCard, centerCard, rightCard, bottomBar) = createRefs()

        val verticalGuideline = createGuidelineFromStart(0.33f)
        val verticalGuideline2 = createGuidelineFromEnd(0.33f)
        val horizontalGuideline = createGuidelineFromTop(0.4f)

        Text(
            text = "Guidelines Layout",
            fontSize = 20.sp,
            fontWeight = FontWeight.Bold,
            modifier = Modifier.constrainAs(title) {
                top.linkTo(parent.top)
                start.linkTo(parent.start)
                end.linkTo(parent.end)
            }
        )

        Card(
            modifier = Modifier
                .fillMaxHeight()
                .constrainAs(leftCard) {
                    top.linkTo(title.bottom, margin = 16.dp)
                    bottom.linkTo(bottomBar.top, margin = 16.dp)
                    start.linkTo(parent.start)
                    end.linkTo(verticalGuideline)
                },
            colors = CardDefaults.cardColors(
                containerColor = Color.Red.copy(alpha = 0.7f)
            )
        ) {
            Box(
                contentAlignment = Alignment.Center,
                modifier = Modifier.fillMaxSize()
            ) {
                Text(
                    text = "Left\n33%",
                    color = Color.White,
                    fontWeight = FontWeight.Bold
                )
            }
        }
    }

    Card(
        modifier = Modifier

```

```

        .fillMaxHeight()
        .constrainAs(centerCard) {
            top.linkTo(title.bottom, margin = 16.dp)
            bottom.linkTo(bottomBar.top, margin = 16.dp)
            start.linkTo(verticalGuideline)
            end.linkTo(verticalGuideline2)
        },
        colors = CardDefaults.cardColors(
            containerColor = Color.Green.copy(alpha = 0.7f)
        )
    ) {
        Box(
            contentAlignment = Alignment.Center,
            modifier = Modifier.fillMaxSize()
        ) {
            Text(
                text = "Center\n34%",
                color = Color.White,
                fontWeight = FontWeight.Bold
            )
        }
    }
}

Card(
    modifier = Modifier
        .fillMaxHeight()
        .constrainAs(rightCard) {
            top.linkTo(title.bottom, margin = 16.dp)
            bottom.linkTo(bottomBar.top, margin = 16.dp)
            start.linkTo(verticalGuideline2)
            end.linkTo(parent.end)
        },
    colors = CardDefaults.cardColors(
        containerColor = Color.Blue.copy(alpha = 0.7f)
    )
) {
    Box(
        contentAlignment = Alignment.Center,
        modifier = Modifier.fillMaxSize()
    ) {
        Text(
            text = "Right\n33%",
            color = Color.White,
            fontWeight = FontWeight.Bold
        )
    }
}
}

```

```

Card(
    modifier = Modifier
        .height(60.dp)
        .constrainAs(bottomBar) {
            bottom.linkTo(parent.bottom)
            start.linkTo(parent.start)
            end.linkTo(parent.end)
        },
    colors = CardDefaults.cardColors(
        containerColor = MaterialTheme.colorScheme.surfaceVariant
    )
) {
    Box(
        contentAlignment = Alignment.Center,
        modifier = Modifier.fillMaxSize()
    ) {
        Text(
            text = "Bottom Navigation Bar",
            fontWeight = FontWeight.Medium
        )
    }
}
}
}

```

GridLayoutExamples.kt

```

package com.example.androidlayoutmanagers
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.grid.GridCells
import androidx.compose.foundation.lazy.grid.LazyVerticalGrid
import androidx.compose.foundation.lazy.grid.items
import androidx.compose.foundation.lazy.grid.rememberLazyGridState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

```

```

data class GridItem(val id: Int, val title: String, val color: Color)

```

```

@Composable

```

```

fun LazyVerticalGridExample() {
    val items = remember {
        (1..50).map { index ->
            val colors = listOf(
                Color.Red, Color.Blue, Color.Green, Color.Yellow,
                Color.Magenta, Color.Cyan, Color.Gray, Color(0xFFFF6B35)
            )
            GridItem(
                id = index,
                title = "Item $index",
                color = colors[index % colors.size]
            )
        }
    }

    val gridState = rememberLazyGridState()

    Column(
        modifier = Modifier.fillMaxSize()
    ) {
        Text(
            text = "LazyVerticalGrid - 2 Columns",
            fontSize = 24.sp,
            fontWeight = FontWeight.Bold,
            modifier = Modifier.padding(16.dp)
        )

        LazyVerticalGrid(
            columns = GridCells.Fixed(2),
            modifier = Modifier.fillMaxSize(),
            state = gridState,
            contentPadding = PaddingValues(16.dp),
            horizontalArrangement = Arrangement.spacedBy(12.dp),
            verticalArrangement = Arrangement.spacedBy(12.dp)
        ) {
            items(items) { item ->
                Card(
                    modifier = Modifier
                        .fillMaxWidth()
                        .aspectRatio(1f),
                    elevation = CardDefaults.cardElevation(defaultElevation = 4.dp)
                ) {
                    Box(
                        modifier = Modifier
                            .fillMaxSize()
                            .background(item.color.copy(alpha = 0.8f)),
                        contentAlignment = Alignment.Center
                    ) {

```

```

        Text(
            text = item.title,
            color = Color.White,
            fontWeight = FontWeight.Bold,
            fontSize = 16.sp
        )
    }
}
}
}
}
}
}
}

```

```

@Composable
fun AdaptiveGridExample() {
    val items = remember {
        (1..30).map { index ->
            val colors = listOf(
                Color(0xFF6200EE), Color(0xFF03DAC6), Color(0xFFCF6679),
                Color(0xFF3700B3), Color(0xFF018786), Color(0xFFBB86FC)
            )
            GridItem(
                id = index,
                title = "Adaptive $index",
                color = colors[index % colors.size]
            )
        }
    }
}

```

```

Column(
    modifier = Modifier.fillMaxSize()
) {
    Text(
        text = "Adaptive Grid - Min 120dp",
        fontSize = 24.sp,
        fontWeight = FontWeight.Bold,
        modifier = Modifier.padding(16.dp)
    )
}

```

```

LazyVerticalGrid(
    columns = GridCells.Adaptive(minSize = 120.dp),
    modifier = Modifier.fillMaxSize(),
    contentPadding = PaddingValues(16.dp),
    horizontalArrangement = Arrangement.spacedBy(8.dp),
    verticalArrangement = Arrangement.spacedBy(8.dp)
) {
    items(items) { item ->
        Card(

```

```

        modifier = Modifier
            .fillMaxWidth()
            .height(100.dp),
        elevation = CardDefaults.cardElevation(defaultElevation = 3.dp)
    ) {
        Box(

```

LazyLayoutExamples.kt

```
package com.example.androidlayoutmanagers
```

```

import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.LazyRow
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.lazy.rememberLazyListState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

```

```
data class Item(val id: Int, val title: String, val description: String)
```

```
@Composable
```

```

fun LazyColumnExample() {
    val items = remember {
        (1..50).map { index ->
            Item(
                id = index,
                title = "Item $index",
                description = "This is the description for item number $index"
            )
        }
    }
}

```

```
val listState = rememberLazyListState()
```

```

Column(
    modifier = Modifier.fillMaxSize()

```

```

    ) {
        Text(
            text = "LazyColumn - ${items.size} items",
            fontSize = 24.sp,
            fontWeight = FontWeight.Bold,
            modifier = Modifier.padding(16.dp)
        )

        LazyColumn(
            modifier = Modifier.fillMaxSize(),
            state = listState,
            contentPadding = PaddingValues(16.dp),
            verticalArrangement = Arrangement.spacedBy(8.dp)
        ) {
            items(items) { item ->
                Card(
                    modifier = Modifier
                        .fillMaxWidth(),
                    elevation = CardDefaults.cardElevation(defaultElevation = 4.dp)
                ) {
                    Column(
                        modifier = Modifier
                            .fillMaxWidth()
                            .padding(16.dp)
                    ) {
                        Text(
                            text = item.title,
                            fontSize = 18.sp,
                            fontWeight = FontWeight.Bold,
                            color = MaterialTheme.colorScheme.primary
                        )
                        Spacer(modifier = Modifier.height(4.dp))
                        Text(
                            text = item.description,
                            fontSize = 14.sp,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    }
                }
            }
        }
    }
}

```

```

@Composable
fun LazyRowExample() {
    val categories = remember {
        listOf
    }
}

```

```

        "Technology", "Sports", "Music", "Movies", "Books",
        "Travel", "Food", "Fashion", "Gaming", "Art",
        "Science", "History", "Nature", "Photography", "Dance"
    )
}

val listState = rememberLazyListState()

Column(
    modifier = Modifier
        .fillMaxSize()
        .padding(16.dp)
) {
    Text(
        text = "LazyRow - Horizontal Scrolling",
        fontSize = 24.sp,
        fontWeight = FontWeight.Bold,
        modifier = Modifier.padding(bottom = 16.dp)
    )

    LazyRow(
        state = listState,
        horizontalArrangement = Arrangement.spacedBy(12.dp),
        contentPadding = PaddingValues(horizontal = 16.dp)
    ) {
        items(categories) { category ->
            Card(
                modifier = Modifier
                    .width(120.dp)
                    .height(80.dp),
                shape = RoundedCornerShape(12.dp),
                elevation = CardDefaults.cardElevation(defaultElevation = 6.dp)
            ) {
                Box(
                    modifier = Modifier
                        .fillMaxSize()
                        .background(
                            Color(
                                red = (0.3f + (category.hashCode() % 100) / 200f),
                                green = (0.5f + (category.hashCode() % 150) / 300f),
                                blue = (0.7f + (category.hashCode() % 200) / 400f)
                            )
                        )
                ),
                contentAlignment = Alignment.Center
            ) {
                Text(
                    text = category,
                    color = Color.White,

```

```

        fontWeight = FontWeight.Bold,
        modifier = Modifier.padding(8.dp)
    )
    }
}
}

Spacer(modifier = Modifier.height(24.dp))

Text(
    text = "Featured Items",
    fontSize = 20.sp,
    fontWeight = FontWeight.Bold,
    modifier = Modifier.padding(bottom = 12.dp)
)

val featuredItems = remember {
    (1..20).map { index ->
        "Featured $index"
    }
}

LazyRow(
    horizontalArrangement = Arrangement.spacedBy(8.dp),
    contentPadding = PaddingValues(horizontal = 16.dp)
) {
    items(featuredItems) { item ->
        Card(
            modifier = Modifier
                .width(150.dp)
                .height(100.dp),
            colors = CardDefaults.cardColors(
                containerColor = MaterialTheme.colorScheme.surfaceVariant
            )
        ) {
            Box(
                contentAlignment = Alignment.Center
            ) {
                Text(
                    text = item,
                    fontWeight = FontWeight.Medium,
                    modifier = Modifier.padding(12.dp)
                )
            }
        }
    }
}
}

```

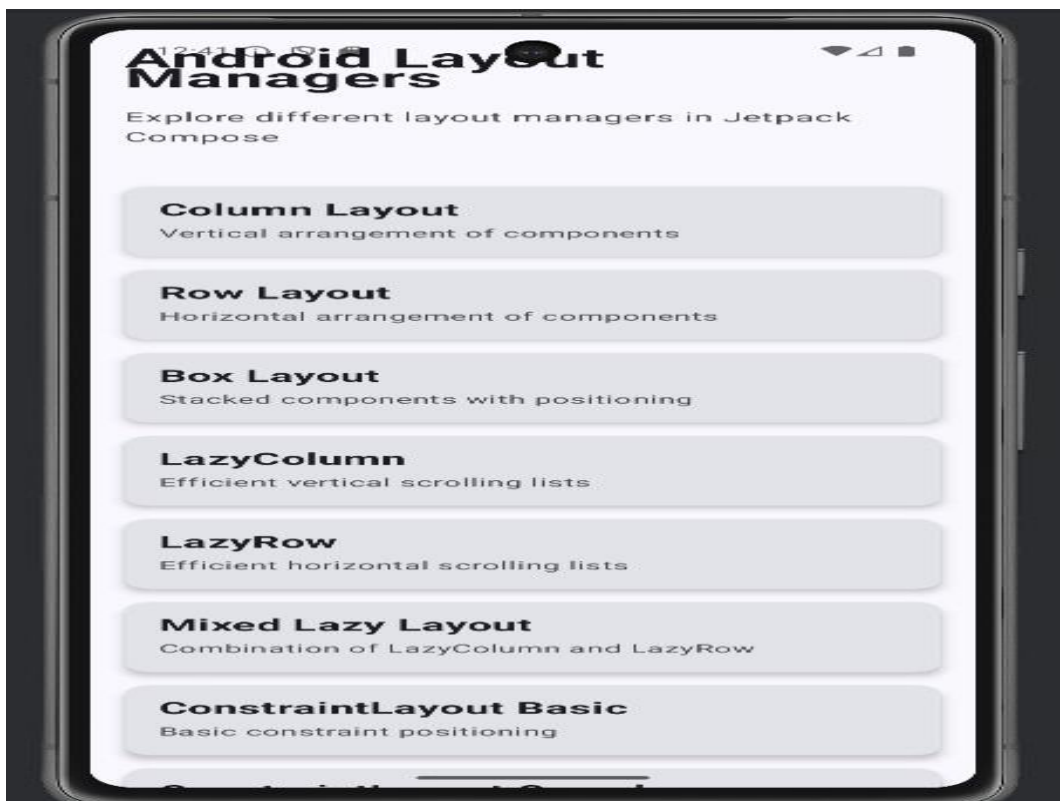


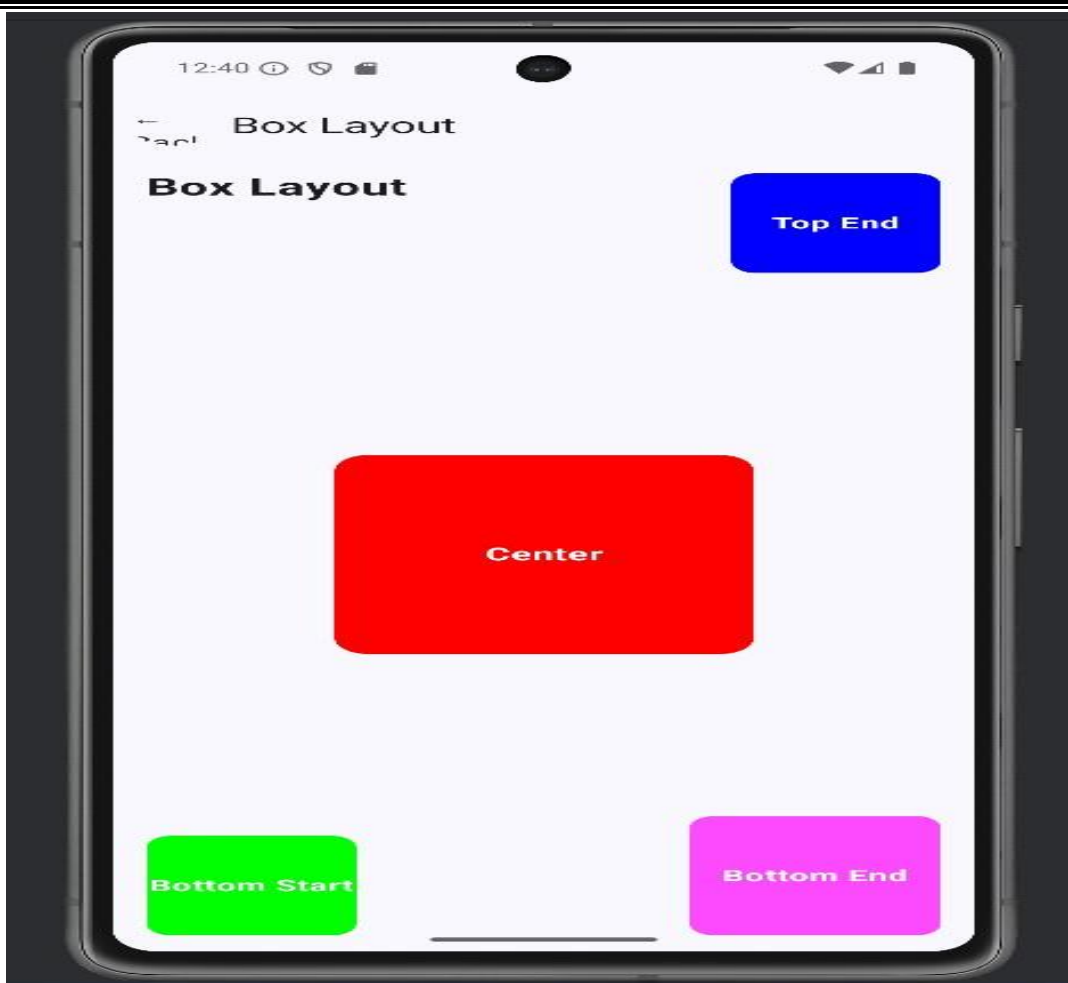
```

        fontWeight = FontWeight.Medium,
        color = MaterialTheme.colorScheme.onPrimaryContainer
    )
    }
}
}
}
} else {
    items(items) { item ->
        Card(
            modifier = Modifier.fillMaxWidth(),
            elevation = CardDefaults.cardElevation(defaultElevation = 2.dp)
        ) {
            Text(
                text = item,
                modifier = Modifier.padding(16.dp)
            )
        }
    }
}
}
}
}
}

```

Output





Expt No : 6

Date:

App Navigation

Aim

To implement navigation between multiple screens in Android app.

Algorithm

1. Create multiple activities/fragments.
2. Use Intent to navigate.
3. Pass data using extras.
4. Implement Navigation component.
5. Test navigation flow.

CODING

MainActivity.kt

```
package com.example.sai1
import android.content.Intent
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Button

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val btnPage1: Button = findViewById(R.id.btnPage1)
        val btnPage2: Button = findViewById(R.id.btnPage2)
        val btnPage3: Button = findViewById(R.id.btnPage3)
        val btnPage4: Button = findViewById(R.id.btnPage4)

        btnPage1.setOnClickListener {
            startActivity(Intent(this, Page1Activity::class.java))
        }

        btnPage2.setOnClickListener {
            startActivity(Intent(this, Page2Activity::class.java))
        }

        btnPage3.setOnClickListener {
            startActivity(Intent(this, Page3Activity::class.java))
        }
    }
}
```

```

        btnPage4.setOnClickListener {
            startActivity(Intent(this, Page4Activity::class.java))
        }
    }
}

```

Activity main.xml

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="16dp"
    android:background="@color/white">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="GUI Components"
        android:textSize="24sp"
        android:textStyle="bold"
        android:layout_marginBottom="40dp" />

    <Button
        android:id="@+id/btnPage1"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Page 1 - Initial State"
        android:textSize="16sp"
        android:layout_marginBottom="16dp" />

    <Button
        android:id="@+id/btnPage2"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Page 2 - Large Font"
        android:textSize="16sp"
        android:layout_marginBottom="16dp" />

    <Button
        android:id="@+id/btnPage3"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Page 3 - Green Text"

```

```
android:textSize="16sp"
android:layout_marginBottom="16dp" />
```

```
<Button
    android:id="@+id/btnPage4"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Page 4 - Blue Background"
    android:textSize="16sp" />
```

```
</LinearLayout>
```

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:supportRtl="true"
        android:theme="@style/Theme.Sai1"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <activity
            android:name=".Page1Activity"
            android:exported="false" />
        <activity
            android:name=".Page2Activity"
            android:exported="false" />
        <activity
            android:name=".Page3Activity"
            android:exported="false" />
        <activity
            android:name=".Page4Activity"
            android:exported="false" />
    </application>
```

</manifest>

OUTPUT



Expt No : 7

Date:

Activity and Fragment Lifecycles

Aim

To understand lifecycle methods of Activity and Fragment.

Algorithm

1. Create activity/fragment.
2. Override lifecycle methods.
3. Use Logcat for logging.
4. Rotate device and observe lifecycle.
5. Analyze lifecycle behavior.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity">
    <TextView
        android:id="@+id/tvTitle"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Student Details"
        android:textAlignment="center"
        android:textSize="24sp" />
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical">
        <fragment
            android:id="@+id/fragmentBasic"
            android:name="org.rajalakshmi.androidfragments.StudentBasicDetails"
            android:layout_width="match_parent"
            android:layout_height="300dp" />
        <fragment
            android:id="@+id/fragmentMark"
            android:name="org.rajalakshmi.androidfragments.StudentMarkDetails"
            android:layout_width="match_parent"
            android:layout_height="300dp" />
    </LinearLayout>
```

</LinearLayout>

fragment_student_basic_details.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".StudentBasicDetails">
    <TextView
        android:id="@+id/tvBasicDetails"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Basic Details"
        android:textAlignment="center"
        android:textSize="24sp" />
    <TextView
        android:id="@+id/tvRegisterNumber"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="50dp"
        android:text="Register No." />
    <EditText
        android:id="@+id/etRegisterNumber"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginLeft="150dp"
        android:layout_marginTop="50dp"
        android:ems="10"
        android:hint="Register Number"
        android:inputType="textPersonName" />
    <TextView
        android:id="@+id/tvName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="125dp"
        android:text="Name" />
    <EditText
        android:id="@+id/etName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginLeft="150dp"
        android:layout_marginTop="125dp"
        android:ems="10"
        android:hint="Name"
        android:inputType="textPersonName" />
```

```

<TextView
android:id="@+id/tvDepartment"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_marginTop="200dp"
android:text="Department" />
<EditText
android:id="@+id/etDepartment"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_marginLeft="150dp"
android:layout_marginTop="200dp"
android:ems="10"
android:hint="Department"
android:inputType="textPersonName" />
</FrameLayout>

```

fragment_student_mark_details.xml

```

<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".StudentMarkDetails">
<TextView
android:id="@+id/tvBasicDetails"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Mark Details"
android:textAlignment="center"
android:textSize="24sp" />
<TextView
android:id="@+id/tvSSLC"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_marginTop="50dp"
android:text="S.S.L.C." />
<EditText
android:id="@+id/etSSLC"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_marginLeft="150dp"
android:layout_marginTop="50dp"
android:ems="10"
android:hint="S.S.L.C. Mark"
android:inputType="textPersonName" />
<TextView
android:id="@+id/tvHSc"

```

```

android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_marginTop="125dp"
android:text="H.Sc." />
<EditText
android:id="@+id/etHSC"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_marginLeft="150dp"
android:layout_marginTop="125dp"
android:ems="10"
android:hint="H.Sc. Mark"
android:inputType="textPersonName" />
<TextView
android:id="@+id/tvUG"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_marginTop="200dp"
android:text="U.G." />
<EditText
android:id="@+id/etUG"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_marginLeft="150dp"
android:layout_marginTop="200dp"
android:ems="10"
android:hint="U.G. C.G.P.A."
android:inputType="textPersonName" />
</FrameLayout>

```

MainActivity.kt

```

package org.rajalakshmi.androidfragments
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}

```

Dept. of Computer Science and Engineering | Rajalakshmi Engineering College 33
StudentBasicDetails.kt

```

package org.rajalakshmi.androidfragments
import android.os.Bundle
import androidx.fragment.app.Fragment
import android.view.LayoutInflater
import android.view.View

```

```

import android.view.ViewGroup
// TODO: Rename parameter arguments, choose names that match
// the fragment initialization parameters, e.g. ARG_ITEM_NUMBER
private const val ARG_PARAM1 = "param1"
private const val ARG_PARAM2 = "param2"
/**
 * A simple [Fragment] subclass.
 * Use the [StudentBasicDetails.newInstance] factory method to
 * create an instance of this fragment.
 */
class StudentBasicDetails : Fragment() {
// TODO: Rename and change types of parameters
private var param1: String? = null
private var param2: String? = null
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
arguments?.let {
param1 = it.getString(ARG_PARAM1)
param2 = it.getString(ARG_PARAM2)
}
}
override fun onCreateView(
inflater: LayoutInflater, container: ViewGroup?,
savedInstanceState: Bundle?
): View? {
// Inflate the layout for this fragment
return inflater.inflate(R.layout.fragment_student_basic_details,
container, false)
}
companion object {
/**
 * Use this factory method to create a new instance of
 * this fragment using the provided parameters.
 *
 * @param param1 Parameter 1.
 * @param param2 Parameter 2.
 * @return A new instance of fragment StudentBasicDetails.
 */
// TODO: Rename and change types and number of parameters
@JvmStatic
fun newInstance(param1: String, param2: String) =
StudentBasicDetails().apply {
arguments = Bundle().apply {
putString(ARG_PARAM1, param1)
putString(ARG_PARAM2, param2)
}
}
}
}

```

```
}
```

```
34 Dept. of Computer Science and Engineering | Rajalakshmi Engineering College
StudentMarkDetails.kt
package org.rajalakshmi.androidfragments
import android.os.Bundle
import androidx.fragment.app.Fragment
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
// TODO: Rename parameter arguments, choose names that match
// the fragment initialization parameters, e.g. ARG_ITEM_NUMBER
private const val ARG_PARAM1 = "param1"
private const val ARG_PARAM2 = "param2"
/**
 * A simple [Fragment] subclass.
 * Use the [StudentMarkDetails.newInstance] factory method to
 * create an instance of this fragment.
 */
class StudentMarkDetails : Fragment() {
// TODO: Rename and change types of parameters
private var param1: String? = null
private var param2: String? = null
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
arguments?.let {
param1 = it.getString(ARG_PARAM1)
param2 = it.getString(ARG_PARAM2)
}
}
override fun onCreateView(
inflater: LayoutInflater, container: ViewGroup?,
savedInstanceState: Bundle?
): View? {
// Inflate the layout for this fragment
return inflater.inflate(R.layout.fragment_student_mark_details,
container, false)
}
companion object {
/**
 * Use this factory method to create a new instance of
 * this fragment using the provided parameters.
 *
 * @param param1 Parameter 1.
 * @param param2 Parameter 2.
 * @return A new instance of fragment StudentMarkDetails.
 */
// TODO: Rename and change types and number of parameters
```

@JvmStatic

```
fun newInstance(param1: String, param2: String) =  
    StudentMarkDetails().apply {  
        arguments = Bundle().apply {  
            putString(ARG_PARAM1, param1)  
            putString(ARG_PARAM2, param2)  
        }  
    }  
}
```

Output

Output



Expt No : 8

Date:

App Architecture (UI Layer)

Aim

To implement UI layer using MVVM architecture.

Algorithm

1. Create ViewModel class.
2. Add LiveData objects.
3. Bind UI with ViewModel.
4. Observe data changes.
5. Update UI automatically.

Coding

Mainactivity.kt

```
package com.example.apparchitectureuilayer

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import com.example.apparchitectureuilayer.presentation.navigation.AppNavigationSimple
import com.example.apparchitectureuilayer.ui.theme.AppArchitectureUILayerTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            AppArchitectureUILayerTheme {
                AppNavigationSimple()
            }
        }
    }
}
```

UserDetailScreen.kt

```
package com.example.apparchitectureuilayer.presentation.screen

import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.material3.*
```

```

import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.hilt.navigation.compose.hiltViewModel
import coil.compose.AsyncImage
import com.example.apparchitectureuilayer.data.model.ApiResponse
import
com.example.apparchitectureuilayer.presentation.viewmodel.UserDetailViewModel

```

```

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun UserDetailScreen(
    userId: String,
    onBack: () -> Unit,
    viewModel: UserDetailViewModel = hiltViewModel()
) {
    LaunchedEffect(userId) {
        viewModel.loadUser(userId)
    }

    val user by viewModel.user.collectAsState()

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("User Details") },
                navigationIcon = {
                    IconButton(onClick = onBack) {
                        Text("←")
                    }
                }
            )
        }
    ) { padding ->
        Box(
            modifier = Modifier
                .fillMaxSize()
                .padding(padding)
        ) {
            when (user) {
                is ApiResponse.Loading -> {
                    CircularProgressIndicator(
                        modifier = Modifier.align(Alignment.Center)
                    )
                }
            }
        }
    }
}

```

```

    }
    is ApiResponse.Success -> {
        val userData = (user as ApiResponse.Success).data
        LazyColumn(
            modifier = Modifier.fillMaxSize(),
            contentPadding = PaddingValues(16.dp),
            verticalArrangement = Arrangement.spacedBy(16.dp)
        ) {
            item {
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.Center
                ) {
                    if (userData.avatar != null) {
                        AsyncImage(
                            model = userData.avatar,
                            contentDescription = "User Avatar",
                            modifier = Modifier
                                .size(120.dp)
                                .clip(CircleShape),
                            contentScale = ContentScale.Crop
                        )
                    } else {
                        Box(
                            modifier = Modifier
                                .size(120.dp)
                                .clip(CircleShape)
                                .background(MaterialTheme.colorScheme.primaryContainer),
                            contentAlignment = Alignment.Center
                        ) {
                            Text(
                                text = userData.name.first().uppercase(),
                                style = MaterialTheme.typography.headlineLarge,
                                color = MaterialTheme.colorScheme.onPrimaryContainer
                            )
                        }
                    }
                }
            }
        }

        item {
            Column(
                horizontalAlignment = Alignment.CenterHorizontally
            ) {
                Text(
                    text = userData.name,
                    style = MaterialTheme.typography.headlineMedium,
                    fontWeight = FontWeight.Bold
                )
            }
        }
    }
}

```

```

    )
    Spacer(modifier = Modifier.height(8.dp))
    Text(
        text = userData.email,
        style = MaterialTheme.typography.bodyLarge,
        color = MaterialTheme.colorScheme.onSurfaceVariant
    )
    Spacer(modifier = Modifier.height(8.dp))
    Text(
        text = if (userData.isActive) "Active" else "Inactive",
        style = MaterialTheme.typography.bodyMedium,
        color = if (userData.isActive)
            MaterialTheme.colorScheme.primary else
            MaterialTheme.colorScheme.error,
        modifier = Modifier
            .padding(horizontal = 16.dp)
            .padding(vertical = 4.dp)
            .background(
                color = if (userData.isActive)
                    MaterialTheme.colorScheme.primaryContainer else
                    MaterialTheme.colorScheme.errorContainer,
                shape = MaterialTheme.shapes.small
            )
    )
    .padding(horizontal = 12.dp, vertical = 4.dp)
    )
}
}

item {
    Card(
        modifier = Modifier.fillMaxWidth()
    ) {
        Column(
            modifier = Modifier
                .fillMaxWidth()
                .padding(16.dp)
        ) {
            Text(
                text = "User Information",
                style = MaterialTheme.typography.titleMedium,
                fontWeight = FontWeight.Bold
            )
            Spacer(modifier = Modifier.height(12.dp))

            InfoRow("ID", userData.id)
            InfoRow("Name", userData.name)
            InfoRow("Email", userData.email)
            InfoRow("Status", if (userData.isActive) "Active" else "Inactive")
        }
    }
}

```



```
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import com.example.apparchitectureuilayer.data.model.User
import com.example.apparchitectureuilayer.presentation.screen.UserListScreenSimple
```

```
@Composable
```

```
fun AppNavigationSimple() {
    var currentUser by remember { mutableStateOf<User?>(null) }
```

```
    if (currentUser == null) {
        UserListScreenSimple(
            onClick = { userId ->
                // Find user and show details
                val users = listOf(
                    User(
                        id = "1",
                        name = "John Doe",
                        email = "john.doe@example.com",
                        avatar = null,
                        isActive = true
                    ),
                    User(
                        id = "2",
                        name = "Jane Smith",
                        email = "jane.smith@example.com",
                        avatar = null,
                        isActive = true
                    ),
                    User(
                        id = "3",
                        name = "Bob Johnson",
                        email = "bob.johnson@example.com",
                        avatar = null,
                        isActive = false
                    ),
                    User(
                        id = "4",
                        name = "Alice Williams",
                        email = "alice.williams@example.com",
                        avatar = null,
                        isActive = true
                    ),
                    User(
```

```

        id = "5",
        name = "Charlie Brown",
        email = "charlie.brown@example.com",
        avatar = null,
        isActive = true
    )
)
currentUser = users.find { it.id == userId }
}
)
} else {
    UserDetailScreenSimple(
        user = currentUser!!,
        onBack = { currentUser = null }
    )
}
}

```

```

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun UserDetailScreenSimple(
    user: User,
    onBack: () -> Unit
) {
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("User Details") },
                navigationIcon = {
                    Button(onClick = onBack) {
                        Text("← Back")
                    }
                }
            )
        }
    ) { padding ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(padding)
                .padding(16.dp),
            verticalArrangement = Arrangement.spacedBy(16.dp)
        ) {
            // User avatar placeholder
            Box(
                modifier = Modifier
                    .size(120.dp)
                    .align(Alignment.CenterHorizontally)
            )
        }
    }
}

```

```

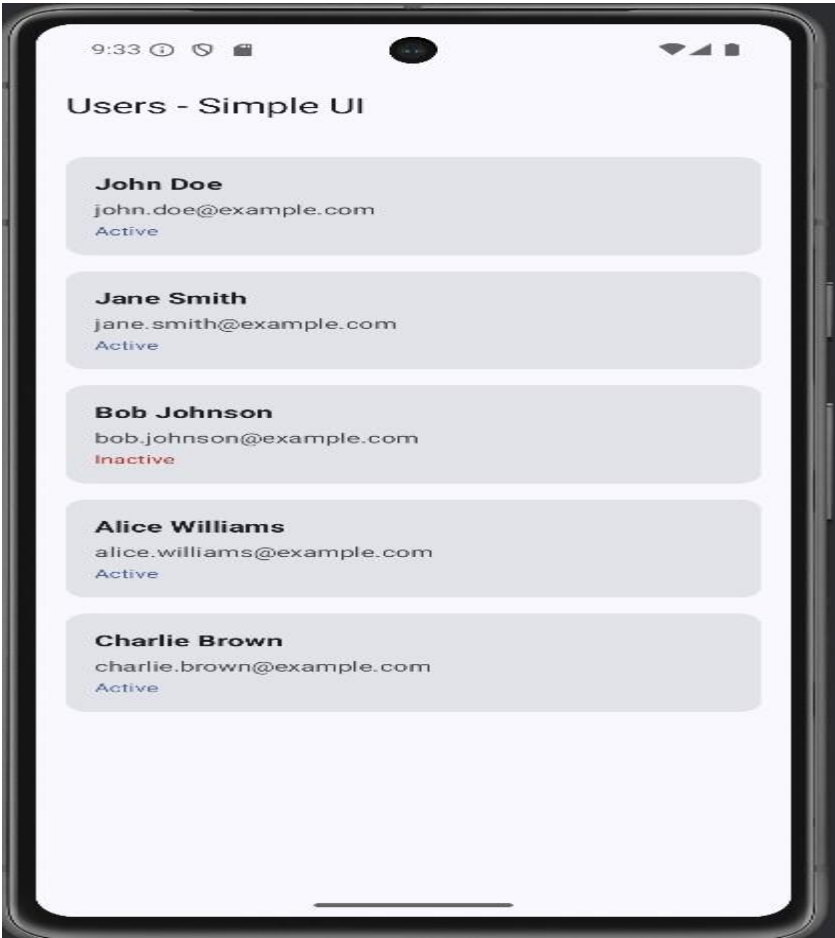
        .clip(RoundedCornerShape(16.dp))
        .background(
            color = MaterialTheme.colorScheme.primaryContainer
        ),
        contentAlignment = Alignment.Center
    ) {
        Text(
            text = user.name.first().uppercase(),
            style = MaterialTheme.typography.headlineLarge,
            color = MaterialTheme.colorScheme.onPrimaryContainer
        )
    }

    // User info
    Card(
        modifier = Modifier.fillMaxWidth()
    ) {
        Column(
            modifier = Modifier
                .fillMaxWidth()
                .padding(16.dp),
            verticalArrangement = Arrangement.spacedBy(8.dp)
        ) {
            Text(
                text = user.name,
                style = MaterialTheme.typography.headlineMedium,
                fontWeight = FontWeight.Bold
            )
            Text(
                text = user.email,
                style = MaterialTheme.typography.bodyLarge,
                color = MaterialTheme.colorScheme.onSurfaceVariant
            )
            Text(
                text = "Status: ${if (user.isActive) "Active" else "Inactive"}",
                style = MaterialTheme.typography.bodyMedium,
                color = if (user.isActive)
                    MaterialTheme.colorScheme.primary else
                    MaterialTheme.colorScheme.error
            )
            Text(
                text = "ID: ${user.id}",
                style = MaterialTheme.typography.bodySmall,
                color = MaterialTheme.colorScheme.onSurfaceVariant
            )
        }
    }
}

```

```
}  
}
```

Output





Expt No : 9

Date:

App Architecture (Persistence)

Aim

To store and retrieve data using Room persistence library.

Algorithm

1. Add Room dependencies.
2. Create Entity class.
3. Create DAO interface.
4. Create Database class.
5. Perform CRUD operations.

MainActivity.kt

```
package com.example.apparchitecturepersistence
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Scaffold
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.compose.runtime.livedata.observeAsState
import com.example.apparchitecturepersistence.data.NoteDatabase
import com.example.apparchitecturepersistence.data.Note
import com.example.apparchitecturepersistence.repository.NoteRepository
import
com.example.apparchitecturepersistence.ui.theme.AppArchitecturePersistenceTheme
import com.example.apparchitecturepersistence.ui.screens.NoteListScreen
import com.example.apparchitecturepersistence.viewmodel.NoteViewModel
import com.example.apparchitecturepersistence.viewmodel.NoteViewModelFactory

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()

        val database = NoteDatabase.getDatabase(this)
        val repository = NoteRepository(database.noteDao())
```

```

setContent {
    AppArchitecturePersistenceTheme {
        Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
            NoteApp(
                repository = repository,
                modifier = Modifier.padding(innerPadding)
            )
        }
    }
}
}

```

@Composable

```

fun NoteApp(repository: NoteRepository, modifier: Modifier = Modifier) {
    val viewModel: NoteViewModel = viewModel(
        factory = NoteViewModelFactory(repository)
    )

```

```

    val notes by viewModel.allNotes.observeAsState(initial = emptyList())
    val selectedNote by viewModel.selectedNote.observeAsState()
    val isLoading by viewModel.isLoading.observeAsState(initial = false)
    val errorMessage by viewModel.errorMessage.observeAsState()

```

NoteListScreen(

```

    notes = notes ?: emptyList(),
    selectedNote = selectedNote,
    isLoading = isLoading ?: false,
    errorMessage = errorMessage,
    onNoteClick = { note -> viewModel.selectNote(note) },
    onNoteDelete = { note -> viewModel.deleteNote(note) },
    onNoteToggle = { note -> viewModel.toggleNoteCompletion(note) },
    onAddNote = { title, content -> viewModel.insertNote(title, content) },
    onUpdateNote = { note -> viewModel.updateNote(note) },
    onClearError = { viewModel.clearError() },
    onDeleteAll = { viewModel.deleteAllNotes() },
    formatTimestamp = { timestamp -> viewModel.formatTimestamp(timestamp) },
    modifier = modifier

```

```

    )
}

```

@Preview(showBackground = true)

@Composable

```

fun NoteAppPreview() {

```

```

    AppArchitecturePersistenceTheme {

```

```

        // Preview would need mock repository for full functionality

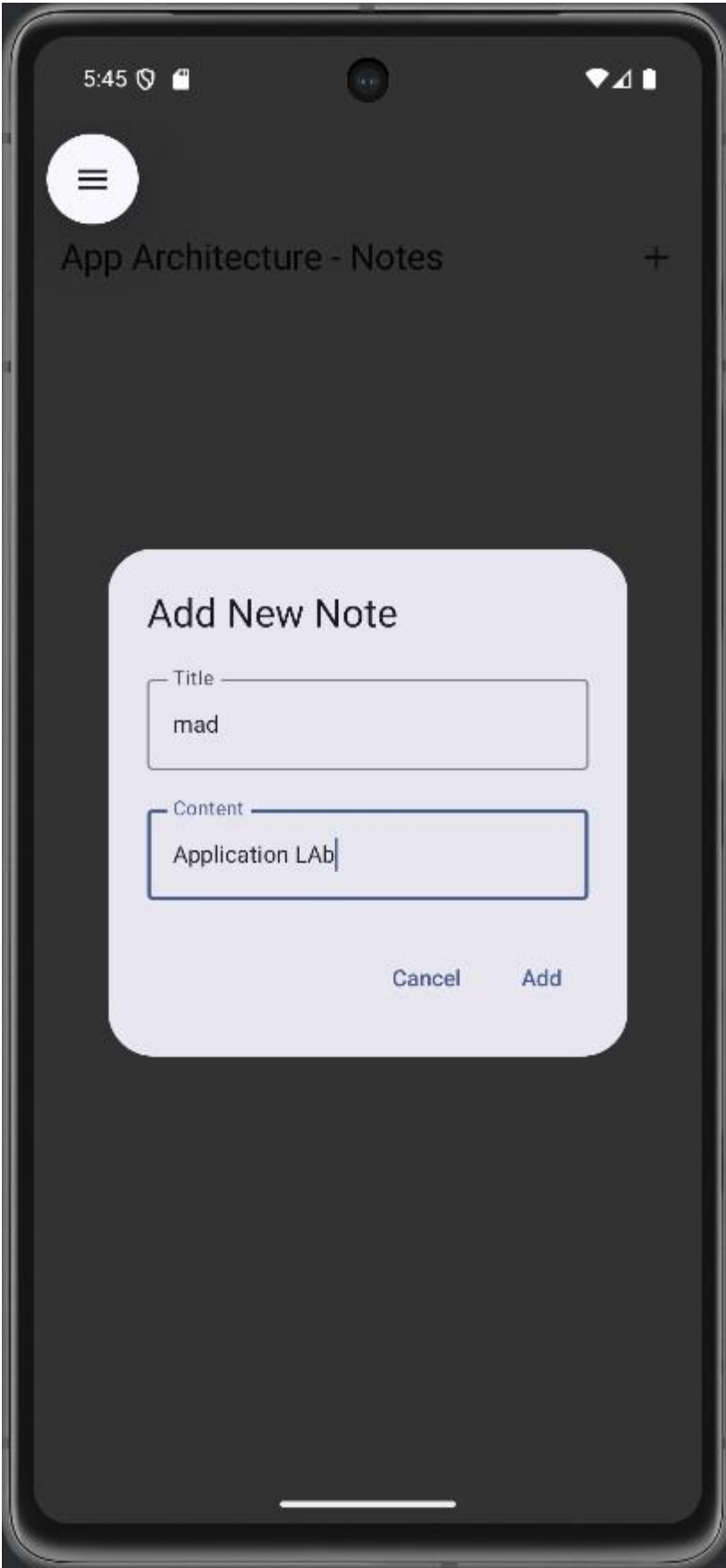
```

```

    }
}

```

Output



Expt No : 10

Date:

Advanced RecyclerView Use Cases

Aim

To display complex and dynamic lists using RecyclerView.

Algorithm

1. Add RecyclerView to layout.
2. Create ViewHolder.
3. Create Adapter.
4. Bind data list.
5. Handle item clicks.

```
package com.example.advancedrecyclerviewusecases
```

```
import android.os.Bundle
```

```
import androidx.appcompat.app.AppCompatActivity
```

```
import androidx.recyclerview.widget.ItemTouchHelper
```

```
import androidx.recyclerview.widget.LinearLayoutManager
```

```
import androidx.recyclerview.widget.RecyclerView
```

```
import
```

```
com.example.advancedrecyclerviewusecases.adapter.AdvancedRecyclerViewAdapter
```

```
import
```

```
com.example.advancedrecyclerviewusecases.helper.ItemTouchHelperCallback
```

```
import com.example.advancedrecyclerviewusecases.model.DataProvider
```

```
import com.example.advancedrecyclerviewusecases.model.Item
```

```
import com.google.android.material.snackbar.Snackbar
```

```
class MainActivity : AppCompatActivity() {
```

```
    private lateinit var recyclerView: RecyclerView
```

```
    private lateinit var adapter: AdvancedRecyclerViewAdapter
```

```
    private lateinit var itemTouchHelper: ItemTouchHelper
```

```
    private var items = DataProvider.getSampleData()
```

```
    override fun onCreate(savedInstanceState: Bundle?) {
```

```
        super.onCreate(savedInstanceState)
```

```
        setContentView(R.layout.activity_main)
```

```
        setupRecyclerView()
```

```
        setupSwipeToDelete()
```

```
        setupDragAndDrop()
```

```
    }
```

```

private fun setupRecyclerView() {
    recyclerView = findViewById(R.id.recyclerView)
    adapter = AdvancedRecyclerViewAdapter(
        items = items,
        onItemClick = { item ->
            handleClick(item)
        },
        onDeleteItem = { item ->
            handleDeleteItem(item)
        }
    )

    recyclerView.apply {
        layoutManager = LinearLayoutManager(this@MainActivity)
        adapter = this@MainActivity.adapter
    }
}

```

```

private fun handleClick(item: Item) {
    Snackbar.make(
        recyclerView,
        "Clicked: ${item.title}",
        Snackbar.LENGTH_SHORT
    ).show()
}

```

```

private fun handleDeleteItem(item: Item) {
    val position = items.indexOf(item)
    if (position != -1) {
        items = items.toMutableList().apply { removeAt(position) }
        adapter.removeItem(item)

        Snackbar.make(
            recyclerView,
            "Deleted: ${item.title}",
            Snackbar.LENGTH_LONG
        ).setAction("Undo") {
            items = items.toMutableList().apply { add(position, item) }
            adapter.updateItems(items)
        }.show()
    }
}

```

```

private fun setupSwipeToDelete() {
    val callback = ItemTouchHelperCallback(
        onItemClick = { fromPosition, toPosition ->
            // Handle item move for drag and drop
        }
    )
}

```

```

        val movedItem = items[fromPosition]
        items = items.toMutableList().apply {
            removeAt(fromPosition)
            add(toPosition, movedItem)
        }
        adapter.notifyItemMoved(fromPosition, toPosition)
    },
    onItemSwiped = { position ->
        // Handle item swipe for delete
        val swipedItem = items[position]
        handleDeleteItem(swipedItem)
    }
)

itemTouchHelper = ItemTouchHelper(callback)
itemTouchHelper.attachToRecyclerView(recyclerView)
}

private fun setupDragAndDrop() {
    // Drag and drop is already set up in setupSwipeToDelete()
    // The ItemTouchHelper handles both swipe and drag operations
}
}

```

Output



Expt No :1 1

Date:

Connect to the Internet

Aim

To fetch data from internet using REST API.

Algorithm

1. Add internet permission.
2. Add Retrofit dependency.
3. Create API interface.
4. Make network call.
5. Display response data.

Coding

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:supportRtl="true"
        android:theme="@style/Theme.SendEmail"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
```

```

tools:context=".MainActivity">
<TextView
android:id="@+id/tvEmail"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="To" />
<EditText
android:id="@+id/etEmail"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:ems="10"
android:inputType="textPersonName" />
<TextView
android:id="@+id/tvSubject"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Subject" />
<EditText
android:id="@+id/etSubject"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:ems="10"
android:inputType="textPersonName" />
<TextView
android:id="@+id/tvMessage"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Message" />
<EditText
android:id="@+id/etMessage"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:ems="10"
android:inputType="textPersonName" />
<Button
android:id="@+id/btSend"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Send"
android:textAllCaps="false" />
</LinearLayout>

```

MainActivity.kt

```

package org.rajalakshmi.sendemail
import android.content.Intent
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle

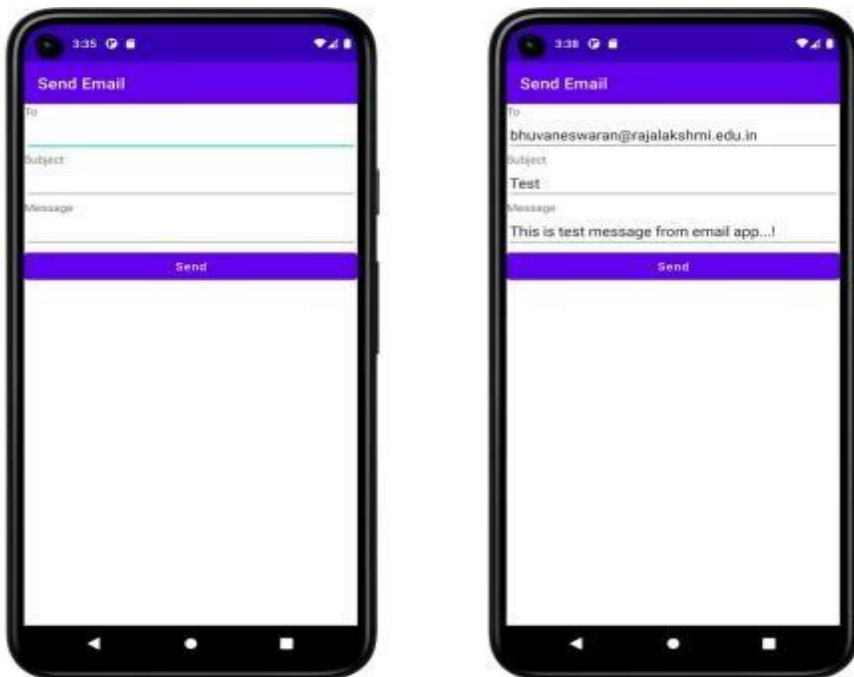
```

```

import android.widget.Button
import android.widget.EditText
import android.widget.TextView
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        val etEmail : EditText = findViewById(R.id.etEmail)
        val etSubject : EditText = findViewById(R.id.etSubject)
        val etMessage : EditText = findViewById(R.id.etMessage)
        val btSend : Button = findViewById(R.id.btSend)
        btSend.setOnClickListener {
            val email = etEmail.text.toString()
            val subject = etSubject.text.toString()
            val message = etMessage.text.toString()
            val intent = Intent(Intent.ACTION_SEND)
            intent.putExtra(Intent.EXTRA_EMAIL, arrayOf(email))
            intent.putExtra(Intent.EXTRA_SUBJECT, subject)
            intent.putExtra(Intent.EXTRA_TEXT, message)
            intent.type = "message/rfc822"
            startActivity(Intent.createChooser(intent, "Choose an Email client
:"))
        }
    }
}

```

Output



Expt No : 12

Date:

Repository Pattern and WorkManager

Aim

To manage data sources and background tasks efficiently.

Algorithm

1. Create repository class.
2. Connect repository to ViewModel.
3. Add WorkManager dependency.
4. Create Worker class.
5. Schedule background task.

Coding

```
package com.example.repository

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import com.example.repository.ui.theme.RepositoryTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            RepositoryTheme {
                RepositoryDemoScreen()
            }
        }
    }
}
```

Output

11:26   
Users refreshed from repository!

Users

Tasks

WorkManager

Refresh Users

Add User

John Doe

john@example.com
555-1234
Synced: Yes

Jane Smith

jane@example.com
555-5678
Synced: No

Bob Johnson

bob@example.com
555-9012
Synced: Yes

New User

new@example.com
123-456-7890
Synced: No

Expt No : 13

Date:

App UI Design

Aim

To design attractive, responsive, and user-friendly Android UI.

Algorithm

1. Apply Material Design principles.
2. Use themes and styles.
3. Add colors and fonts.
4. Use icons and animations.
5. Test UI responsiveness.

MainActivity.kt

```
package com.example.japp9
```

```
import android.os.Bundle
import android.os.Handler
import android.os.Looper
import android.text.Editable
import android.text.TextWatcher
import android.widget.Button
import android.widget.EditText
import androidx.appcompat.app.AppCompatActivity
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView
import com.example.japp9.adapter.AdvancedRecyclerViewAdapter
import com.example.japp9.model.ExpandableItemModel
import com.example.japp9.model.RecyclerViewItem
import com.example.japp9.model.User
import com.example.japp9.utils.DragAndDropCallback
import com.example.japp9.utils.FilterUtils
import com.example.japp9.utils.InfiniteScrollListener
import com.example.japp9.utils.SwipeToDeleteCallback
```

```
class MainActivity : AppCompatActivity() {
```

```
    private lateinit var recyclerView: RecyclerView
    private lateinit var adapter: AdvancedRecyclerViewAdapter
    private lateinit var searchEditText: EditText
    private lateinit var toggleSwipeButton: Button
```

```
private lateinit var toggleDragButton: Button
```

```
private var allItems = mutableListOf<RecyclerViewItem>()
```

```
private var currentItems = mutableListOf<RecyclerViewItem>()
```

```
private var swipeToDeleteHelper: androidx.recyclerview.widget.ItemTouchHelper? =  
null
```

```
private var dragAndDropHelper: androidx.recyclerview.widget.ItemTouchHelper? =  
null
```

```
private var infiniteScrollListener: InfiniteScrollListener? = null
```

```
private var isLoading = false
```

```
private var currentPage = 1
```

```
private val itemsPerPage = 10
```

```
override fun onCreate(savedInstanceState: Bundle?) {
```

```
    super.onCreate(savedInstanceState)
```

```
    setContentView(R.layout.activity_main)
```

```
    initViews()
```

```
    setupRecyclerView()
```

```
    generateInitialData()
```

```
    setupListeners()
```

```
    setupGestures()
```

```
}
```

```
private fun initViews() {
```

```
    recyclerView = findViewById(R.id.recyclerView)
```

```
    searchText = findViewById(R.id.searchEditText)
```

```
    toggleSwipeButton = findViewById(R.id.toggleSwipeButton)
```

```
    toggleDragButton = findViewById(R.id.toggleDragButton)
```

```
}
```

```
private fun setupRecyclerView() {
```

```
    adapter = AdvancedRecyclerViewAdapter(  
        items = currentItems,
```

```
        onItemClick = { item ->
```

```
            handleClick(item)
```

```
    },
```

```
    onDeleteItem = { position ->
```

```
        adapter.removeItem(position)
```

```
    }
```

```
)
```

```
recyclerView.layoutManager = LinearLayoutManager(this)
```

```
recyclerView.adapter = adapter
```

```
// Setup infinite scrolling
```

```
val layoutManager = recyclerView.layoutManager as LinearLayoutManager
```

```

infiniteScrollListener = object : InfiniteScrollListener(layoutManager) {
    override fun onLoadMore() {
        loadMoreItems()
    }
}
recyclerView.addOnScrollListener(infiniteScrollListener!!)
}

private fun generateInitialData() {
    // Generate sample users
    val users = (1..20).map { index ->
        User(
            id = index,
            name = "User $index",
            email = "user$index@example.com",
            avatar = "",
            isActive = index % 2 == 0
        )
    }

    // Generate sample expandable items
    val expandableItems = (1..5).map { index ->
        ExpandableItemModel(
            id = index + 100,
            title = "Category $index",
            description = "Description for category $index",
            subItems = (1..3).map { subIndex -> "Sub item $subIndex in category $index" }
        )
    }

    allItems.clear()
    allItems.add(RecyclerViewItem.HeaderItem("Active Users"))
    users.filter { it.isActive }.take(5).forEach {
        allItems.add(RecyclerViewItem.UserItem(it))
    }

    allItems.add(RecyclerViewItem.HeaderItem("Categories"))
    expandableItems.forEach {
        allItems.add(RecyclerViewItem.ExpandableItem(it))
    }

    allItems.add(RecyclerViewItem.HeaderItem("All Users"))
    users.forEach {
        allItems.add(RecyclerViewItem.UserItem(it))
    }

    currentItems.clear()
    currentItems.addAll(allItems.take(itemsPerPage))
}

```

```

        adapter.updateItems(currentItems)
    }

    private fun setupListeners() {
        searchText.addTextChangedListener(object : TextWatcher {
            override fun beforeTextChanged(s: CharSequence?, start: Int, count: Int, after: Int) {}
            override fun onTextChanged(s: CharSequence?, start: Int, before: Int, count: Int) {}
            override fun afterTextChanged(s: Editable?) {
                filterItems(s?.toString() ?: "")
            }
        })

        toggleSwipeButton.setOnClickListener {
            toggleSwipeToDelete()
        }

        toggleDragButton.setOnClickListener {
            toggleDragAndDrop()
        }
    }

    private fun setupGestures() {
        // Setup swipe to delete
        val swipeCallback = SwipeToDeleteCallback { position ->
            adapter.removeItem(position)
        }
        swipeToDeleteHelper =
        androidx.recyclerview.widget.ItemTouchHelper(swipeCallback)
        swipeToDeleteHelper?.attachToRecyclerView(recyclerView)

        // Setup drag and drop
        val dragCallback = DragAndDropCallback { fromPosition, toPosition ->
            adapter.moveItem(fromPosition, toPosition)
        }
        dragAndDropHelper =
        androidx.recyclerview.widget.ItemTouchHelper(dragCallback)
        // Don't attach by default, let user toggle it
    }

    private fun handleItemClick(item: RecyclerViewItem) {
        when (item) {
            is RecyclerViewItem.ExpandableItem -> {
                // Toggle expandable item
                val updatedItem = item.item.copy(isExpanded = !item.item.isExpanded)
                val index = currentItems.indexOf(item)
                if (index != -1) {

```

```

        currentItems[index] = RecyclerViewItem.ExpandableItem(updatedItem)
        adapter.notifyItemChanged(index)
    }
}
is RecyclerViewItem.UserItem -> {
    // Handle user click - could show details or edit
    println("Clicked on user: ${item.user.name}")
}
else -> {}
}
}

private fun filterItems(query: String) {
    val filtered = FilterUtils.filterItems(allItems, query)
    currentItems.clear()
    currentItems.addAll(filtered.take(itemsPerPage))
    adapter.updateItems(currentItems)
}

private fun toggleSwipeToDelete() {
    if (swipeToDeleteHelper == null) {
        val swipeCallback = SwipeToDeleteCallback { position ->
            adapter.removeItem(position)
        }
        swipeToDeleteHelper =
androidx.recyclerview.widget.ItemTouchHelper(swipeCallback)
        swipeToDeleteHelper?.attachToRecyclerView(recyclerView)
        toggleSwipeButton.text = "Disable Swipe"
    } else {
        swipeToDeleteHelper?.attachToRecyclerView(null)
        swipeToDeleteHelper = null
        toggleSwipeButton.text = "Enable Swipe"
    }
}

private fun toggleDragAndDrop() {
    if (dragAndDropHelper == null) {
        val dragCallback = DragAndDropCallback { fromPosition, toPosition ->
            adapter.moveItem(fromPosition, toPosition)
        }
        dragAndDropHelper =
androidx.recyclerview.widget.ItemTouchHelper(dragCallback)
        dragAndDropHelper?.attachToRecyclerView(recyclerView)
        toggleDragButton.text = "Disable Drag"
    } else {
        dragAndDropHelper?.attachToRecyclerView(null)
        dragAndDropHelper = null
        toggleDragButton.text = "Enable Drag"
    }
}

```

```

    }
}

private fun loadMoreItems() {
    if (isLoading) return

    isLoading = true

    // Add loading indicator
    val loadingList = currentItem.toMutableList()
    loadingList.add(RecyclerViewItem.LoadingItem())
    adapter.updateItems(loadingList)

    // Simulate network delay
    Handler(Looper.getMainLooper()).postDelayed({
        val startIndex = currentPage * itemsPerPage
        val endIndex = (startIndex + itemsPerPage).coerceAtMost(allItems.size)

        if (startIndex < allItems.size) {
            currentItem.addAll(allItems.subList(startIndex, endIndex))
            currentPage++

            // Remove loading indicator and add new items
            adapter.updateItems(currentItem)
            infiniteScrollListener?.setLoaded()
        }

        isLoading = false
    }, 1500)
}
}

```

